# Mastering PostgreSQL Performance: A Systematic Approach to Query Tuning

**Senior Database Consultant**

Janis Griffin

Quest®

Where Next Meets Now.

# Who Am I?

**Current – 30+ Years in Oracle®, DB2®, ASE, SQL Server®, MySQL®, PostgreSQL**

**DBA and Developer**

- Specialize in Performance Tuning
- Customers Common Question: How do I tune it?

Oracle ACE Pro

**Janis.Griffin@quest.com**

**X - @DoBoutAnything**

Quest®
Where Next Meets Now.

# Agenda

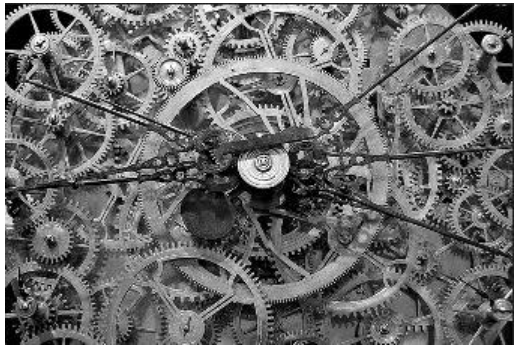
Quest

- **Challenges of Tuning**
- **My 5 Tips**
  - Monitor Wait Time
  - Review the Explain Plan
  - Gather Object Information
  - Find the Driving Table
  - Engineer out the Stupid
- **Several Case Studies**

# Challenges of Tuning

Quest

- **SQL Tuning is Hard**
  - Who should tune – DBA or Developer
  - Which SQL to tune
- **Requires Expertise in Many Areas**
  - Technical – Plan, Data Access, SQL Design
  - Business – What is the Purpose of SQL?
- **Tuning Takes Time**
  - Large Number of SQL Statements
  - Each Statement is Different
- **Low Priority in Some Companies**
  - Vendor Applications
  - Focus on Hardware or System Issues
- **Never Ending**



# Monitor Wait Time – All levels

Quest

Foglight Cloud by Quest | Preprod | Search

Databases > PostgreSQL Server: jgss2017.bglx5qxmyysubhgtflk5hdddye.gx.internal.cloudapp.net:5432 > SQL PI

Time Period: Wednesday, Aug 28, 2024 03:58 PM - 10:01 PM 6 hours | Reports

PostgreSQL-PI@jgss2017.bglx5qxmyysubhgtflk5hdddye.gx.internal.cloudapp.net_5432

Overview | SQL PI | Databases | Tables | Connections | BG Writer | Locks | Configuration | Tablespaces | Replication

Instance Top 50

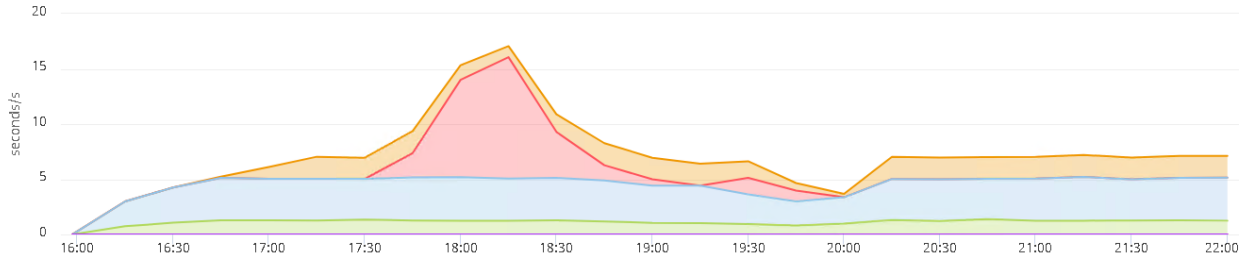
- Statements
- Databases
- Users
- Client Machines
- Programs
- Sessions
- Backend Type

Workload | Client 20% | I/O 47% | IPC 0% | LWLock 0% | Lock 17% | Other 0% | Process Time 16%

PostgreSQL-PI@jgss2017.bglx5qxmyysubhgtflk5hdddye.g...

Resource Breakdown | Breakdown | Baseline

Explore top wait events

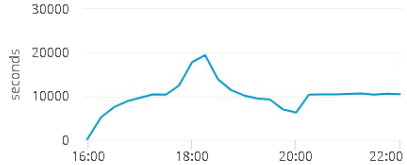


| | |
|---|---|
| IPC Wait | 0.05% |
| LWLock Wait | 0.01% |
| I/O Wait | 47.2% |
| Client Wait | 19.78% |

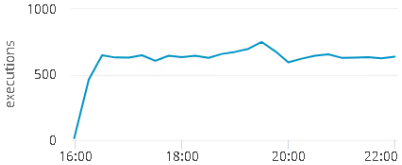
1/2

Hide Legend

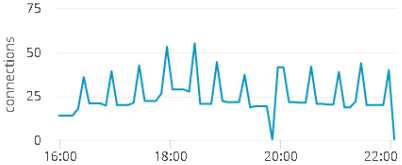
Elapsed Time




Executions




Used Connections




Instance Information

Availability: 100%

| Name | Type |
|---|---|
| jgss2017a... | PostgreSQL |
| Version | Host |
| 16.2 | jgss2017.b... |

Workload Metrics

# Monitor Wait Time – Database Level

Quest

Databases > ... > Statements for Postgresqlagent > PostgreSQL Databases

Tuesday, Apr 2, 2024 02:44:01 PM - Now 1 hour | Reports

Overview | Databases | Tables | Connections | BG Writer | Locks | Configuration | Statements | Tablespaces

Search

| Health | Name | Connections | Tables | Functions | Workload | Blk Hit Rate | Blk Read Rate | Blk Hit % | DB Size | Conflicts Rate | Age | Tablespace | Statements | Admin |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| | test | 3 | 5 | 1 | n/a | 2.1 K/s | 0.6 c/s | 100 % | 29.7 MB | 0 c/s | 439 count | pg_default | 3 | |
| | dvdrental | 0 | 15 | 0 | n/a | 20.1 c/s | 0.7 c/s | 96.7 % | 17.9 MB | 0 c/s | 439 count | pg_default | 2 | |
| | postgres | 1 | 0 | 0 | n/a | 21 c/s | 0.8 c/s | 96.3 % | 7.7 MB | 0 c/s | 439 count | pg_default | 13 | |
| | dot | 3 | 8 | 3 | n/a | 17.3 K/s | 8 K/s | 68.5 % | 907.4 MB | 0 c/s | 439 count | pg_default | 16 | |

DB Summary for test - OID 16718 - Stats Reset 3/25/24, 9:30 PM

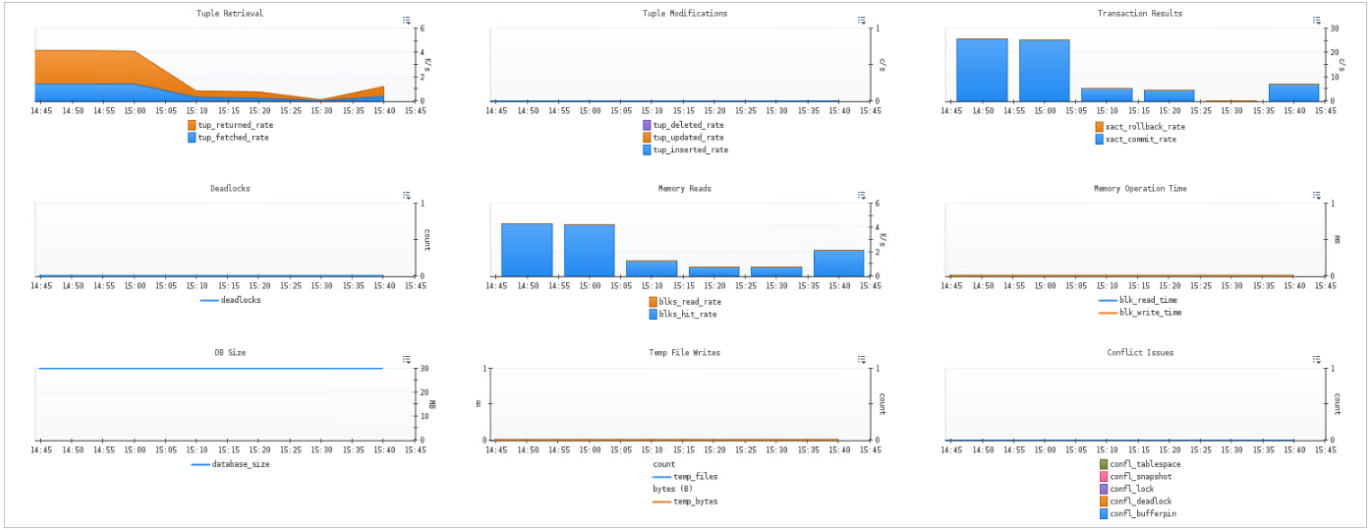
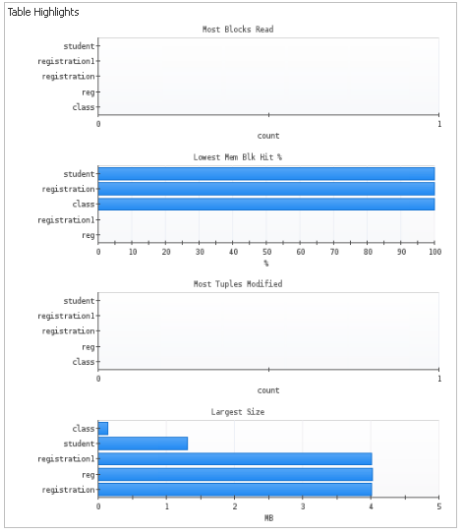



Table Highlights



# Monitor Wait Time – Table Level

Quest

Databases > ... > Statements for Postgresqlagent > Tables

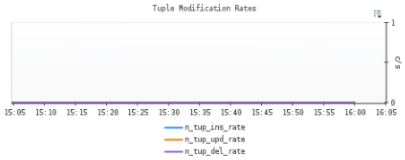
Tuesday, Apr 2, 2024 03:03:40 PM - Now 1 hour | Reports

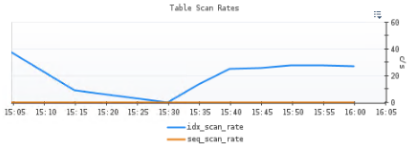
Overview | Databases | Tables | Connections | BG Writer | Locks | Configuration | Statements | Tablespaces

Select Databases

| Health | Name | DB | Schema | Tuples | Indexes | Memory Hit % | Idx Mem Hit % | Idx/Seq Scan Tuple % | HOT Update % | Idx Size | Table Size | Admin |
|---|---|---|---|---|---|---|---|---|---|---|---|---|
| | country | dvdrental | public | 109 | 1 | 0 % | 0 % | 0 % | 0 % | 16 KB | 40 KB | |
| | customer | dvdrental | public | 599 | 4 | 0 % | 0 % | 0 % | 0 % | 152 KB | 96 KB | |
| | film | dvdrental | public | 1 K | | | | | | | | |
| | film_actor | dvdrental | public | 5.5 K | | | | | | | | |
| | film_category | dvdrental | public | 1 K | | | | | | | | |
| | inventory | dvdrental | public | 4.6 K | | | | | | | | |
| | language | dvdrental | public | 6 | | | | | | | | |
| | payment | dvdrental | public | 14.6 K | | | | | | | | |
| | rental | dvdrental | public | 16 K | | | | | | | | |
| | staff | dvdrental | public | 2 | | | | | | | | |
| | store | dvdrental | public | 2 | | | | | | | | |
| | class | test | test | 1 K | | | | | | | | |
| | reg | test | test | 80 K | | | | | | | | |
| | registration | test | test | 80 K | | | | | | | | |
| | registration1 | test | test | 80 K | | | | | | | | |
| | student | test | test | 10 K | | | | | | | | |

Table Summary for registration - OID 16729




DB: test

Possible Need for Indexes

## Possible Need for Indexes

### Name

Possible Need for Indexes

### Description

As a table grows, read queries that use sequential scans (scanning the entire table) will become more costly. If a large amount of the data is returned, sequential scans may still be the most efficient method. For retrieving a small percentage of data from a large table, index scans are more efficient. In this case, if an index exists which can serve the filtering parameters of the read query, the index is scanned first for matching rows which are then retrieved from the table. The below tables have been identified as large tables with the lowest index to sequence scan percentages.

| DB | Table | Live_Tuples | Idx_Scan_Pct | Idx_Scans | Seq_Scans |
|---|---|---|---|---|---|
| test | registration | 79981.0 | 1.4242115971515767 | 28.0 | 1938.0 |

### Next Steps

Before creating a new index, first look at the queries being run on these tables. Running the query with EXPLAIN will show what kind of scan is being performed when the query is run. If a query seems like it would be improved by doing an index scan, look at the columns in the query that are being used to filter for matching rows and try adding them to an index.

If all of the columns being requested are present in the index, the query will operate even more efficiently because it will not be necessary to read from the table. However, indexes also take up space and are scanned during an index scan, so

# Monitor Wait Time – Statement Level

Quest

Databases > ... > PostgreSQL Databases > SQL PI

Time Period: Wednesday, Aug 28, 2024 03:33 PM - 10:48 PM 7.2 hours | Reports

PostgreSQL-PI@jgss2017.bglx5qxmyysubhgtflk5hdddye.gx.internal.cloudapp.net_5432 | Overview | SQL PI | Databases | Tables | Connections | BG Writer | Locks | Configuration | Tablespaces | Replication

**Instance** Top 25

- Statements
  - SELECT t.relid as reli...
  - alter table t_ontime ...
  - CALL public.get_pop...
  - SELECT i.relid as tabl...
  - drop index pk_city_...
- Databases
- Users
- Client Machines
- Programs
- Sessions
- Backend Type

Workload | Client 20% | I/O 47% | IPC 0% | LWLock 0% | Lock 17% | Other 0% | Process Time 16%

PostgreSQL-PI@jgss2017.bglx5qxmyysubhgtflk5hdddye....

## Blocked Sessions

Duration

Search

| Start Date | Session | Blocked By | Status | Duration (Sec) | Program | DB User | SQL Text |
|---|---|---|---|---|---|---|---|
| 08/28/2024 1:00:00 PM | 2156 (2024-... | | Blocking | 1.24M | psql | postgres | fetch all in "pop_flights"; |
| 08/28/2024 1:00:00 ... | 9220 (2024-... | 2156 | Blocked | 1.24M | psql | postgres | alter table t_ontime drop constrai... |
| 08/28/2024 12:00:00 PM | 9788 (2024-... | | Blocking | 916.28K | psql | postgres | fetch all in "pop_flights"; |
| 08/28/2024 12:00:00... | 9220 (2024-... | 9788 | Blocked | 916.28K | psql | postgres | alter table t_ontime drop constrai... |
| 08/28/2024 1:00:00 PM | 9788 (2024-... | | Blocking | 621.24K | psql | postgres | fetch all in "pop_flights"; |
| 08/28/2024 1:00:00 ... | 9220 (2024-... | 9788 | Blocked | 621.24K | psql | postgres | alter table t_ontime drop constrai... |
| 08/28/2024 12:00:00 PM | 2156 (2024-... | | Blocking | 615.8K | psql | postgres | fetch all in "pop_flights"; |
| 08/28/2024 12:00:00... | 9220 (2024-... | 2156 | Blocked | 615.8K | psql | postgres | alter table t_ontime drop constrai... |
| 08/28/2024 2:00:00 PM | 2156 (2024-... | | Blocking | 370.38K | psql | postgres | fetch all in "pop_flights"; |
| 08/28/2024 2:00:00 ... | 9220 (2024-... | 2156 | Blocked | 370.38K | psql | postgres | drop index pk_city_market; |

**Instance Information**

Availability: 100%

| Name | Type |
|---|---|
| jgss2017a... | PostgreSQL |
| **Version** | **Host** |
| 16.2 | jgss2017.b... |

# Wait Events

Quest

- [RDS for PostgreSQL wait events]
- [Aurora PostgreSQL wait events]
- https://www.postgresql.org/docs/current/monitoring-stats
  - Blocking Locks Query

```
select pid,
       usename,
       pg_blocking_pids(pid) as blocked_by,
       query as blocked_query
from pg_stat_activity
where cardinality(pg_blocking_pids(pid)) > 0;
```

```
dot=# \i pg_lock.sql
 pid  | usename   | blocked_by |                          blocked_query
------+-----------+------------+-----------------------------------------------------------------------------
 6352 | postgres  | {10228}    | update public.t_ontime set day_of_week = 9 where day_of_week =1;
(1 row)
```

  - Waiting to read data from the client (either too much data or client is slow)

```
select datname, pid, usename,  application_name, wait_event,
        wait_event_type, query_start, state_change, state, query
from pg_catalog.pg_stat_activity;
```

```
dot=# \i pg_wait.sql
 datname | pid  | usename  | application_name |     wait_event      | wait_event_type |          query_start          |          state_change         |        state        |
                query
---------+------+----------+------------------+---------------------+-----------------+-------------------------------+-------------------------------+---------------------+-----------------------------
---------------------------------------------------------------------------------------------
         | 3356 |          |                  | AutoVacuumMain      | Activity        |                               |                               |                     |
         | 2892 | postgres |                  | LogicalLauncherMain | Activity        |                               |                               |                     |
 dot     | 9716 | postgres | psql             | ClientRead          | Client          | 2024-04-15 21:04:34.452726+00 | 2024-04-15 21:04:34.472967+00 | idle in transaction | fetch all in "pop_flights";
```

# Lock_Monitor View to See Blockers & Waiters

Stackoverflow - lock_monitor

```
CREATE VIEW public.lock_monitor AS(
SELECT
    COALESCE(blockingl.relation::regclass::text,blockingl.locktype) as locked_item,
    now() - blockeda.query_start AS waiting_duration, blockeda.pid AS blocked_pid,
    blockeda.query as blocked_query, blockedl.mode as blocked_mode,
    blockinga.pid AS blocking_pid, blockinga.query as blocking_query,
    blockingl.mode as blocking_mode
 FROM pg_catalog.pg_locks blockedl
JOIN pg_stat_activity blockeda ON blockedl.pid = blockeda.pid
JOIN pg_catalog.pg_locks blockingl ON(
      ( (blockingl.transactionid=blockedl.transactionid) OR
        (blockingl.relation=blockedl.relation AND blockingl.locktype=blockedl.locktype)
      ) AND blockedl.pid != blockingl.pid)
JOIN pg_stat_activity blockinga ON blockingl.pid = blockinga.pid
   AND blockinga.datid = blockeda.datid
 WHERE NOT blockedl.granted
 AND blockinga.datname = current_database() );

 SELECT * from lock_monitor;
```

```
SELECT locked_item, blocked_pid, blocking_pid, blocked_mode,
     substr(blocked_query, 1, 30) blocked_query,
     substr(blocking_query, 1,30) blocking_query
FROM lock_monitor;
```

# Lock_Monitor View - Example

Quest

```
dot=# select locked_item, blocked_pid, blocking_pid, blocked_mode,
dot-#  substr(blocked_query, 1, 30) blocked_query, substr(blocking_query, 1,30) blocking_query
dot-# from lock_monitor;
```

| locked item | blocked pid | blocking pid | blocked mode | blocked query | blocking query |
|---|---|---|---|---|---|
| account | 2672 | 8868 | AccessShareLock | SELECT t.relid as relid, t.sch | SELECT t.relid as relid, t.sch |
| account | 1632 | 8868 | AccessShareLock | SELECT t.relid as relid, t.sch | SELECT t.relid as relid, t.sch |
| account | 680 | 8868 | AccessShareLock | SELECT t.relid as relid, t.sch | SELECT t.relid as relid, t.sch |
| account | 4524 | 8868 | AccessShareLock | SELECT t.relid as relid, t.sch | SELECT t.relid as relid, t.sch |
| account | 916 | 8868 | AccessExclusiveLock | alter table account add column | SELECT t.relid as relid, t.sch |
| account | 6384 | 8868 | AccessShareLock | select * from account where ac | SELECT t.relid as relid, t.sch |
| account | 2672 | 6384 | AccessShareLock | SELECT t.relid as relid, t.sch | select * from account where ac |
| account | 1632 | 6384 | AccessShareLock | SELECT t.relid as relid, t.sch | select * from account where ac |
| account | 680 | 6384 | AccessShareLock | SELECT t.relid as relid, t.sch | select * from account where ac |
| account | 4524 | 6384 | AccessShareLock | SELECT t.relid as relid, t.sch | select * from account where ac |
| account | 916 | 6384 | AccessExclusiveLock | alter table account add column | select * from account where ac |
| account | 8868 | 6384 | AccessShareLock | SELECT t.relid as relid, t.sch | select * from account where ac |
| account | 2672 | 8156 | AccessShareLock | SELECT t.relid as relid, t.sch | select * from public.account; |
| account | 1632 | 8156 | AccessShareLock | SELECT t.relid as relid, t.sch | select * from public.account; |
| account | 680 | 8156 | AccessShareLock | SELECT t.relid as relid, t.sch | select * from public.account; |
| account | 4524 | 8156 | AccessShareLock | SELECT t.relid as relid, t.sch | select * from public.account; |
| account | 916 | 8156 | AccessExclusiveLock | alter table account add column | select * from public.account; |
| account | 6384 | 8156 | AccessShareLock | select * from account where ac | select * from public.account; |
| account | 8868 | 8156 | AccessShareLock | SELECT t.relid as relid, t.sch | select * from public.account; |
| account | 2672 | 916 | AccessShareLock | SELECT t.relid as relid, t.sch | alter table account add column |
| account | 1632 | 916 | AccessShareLock | SELECT t.relid as relid, t.sch | alter table account add column |
| account | 680 | 916 | AccessShareLock | SELECT t.relid as relid, t.sch | alter table account add column |
| account | 4524 | 916 | AccessShareLock | SELECT t.relid as relid, t.sch | alter table account add column |
| account | 6384 | 916 | AccessShareLock | select * from account where ac | alter table account add column |
| account | 8868 | 916 | AccessShareLock | SELECT t.relid as relid, t.sch | alter table account add column |
| account | 2672 | 4524 | AccessShareLock | SELECT t.relid as relid, t.sch | SELECT t.relid as relid, t.sch |
| account | 1632 | 4524 | AccessShareLock | SELECT t.relid as relid, t.sch | SELECT t.relid as relid, t.sch |
| account | 680 | 4524 | AccessShareLock | SELECT t.relid as relid, t.sch | SELECT t.relid as relid, t.sch |
| account | 916 | 4524 | AccessExclusiveLock | alter table account add column | SELECT t.relid as relid, t.sch |

# Review the Explain Plan

- **EXPLAIN - explain description & options**
  - Gives estimated costs (start_up / total cost) as it doesn't actually run it

```
dvdrental=# explain select * from film where title like '%Braveheart%';
                        QUERY PLAN
-------------------------------------------------------
 Seq Scan on film  (cost=0.00..76.50 rows=1 width=384)
   Filter: ((title)::text ~~ '%Braveheart%'::text)
(2 rows)
```

- **EXPLAIN analyze - what is explain cost**
  - Executes the query so actual run time statistics are shown

```
dvdrental=# explain analyze select * from film where title like '%Braveheart%';
                                          QUERY PLAN
-----------------------------------------------------------------------------------------------
 Seq Scan on film  (cost=0.00..76.50 rows=1 width=384) (actual time=0.064..0.421 rows=1 loops=1)
   Filter: ((title)::text ~~ '%Braveheart%'::text)
   Rows Removed by Filter: 999
 Planning Time: 0.157 ms
 Execution Time: 0.450 ms
(5 rows)
```

# Examine the Explain Plan

- **Find Expensive Operators**
  - Examine costs & row counts (each step shows the # of rows returned in step – not what is evaluated)
    - Need to also look at **# rows removed**
    - Cost gives an estimate of resources used (CPU and disk I/O)
  - Look at Seq Scans & Index Scans
- **Review the Filter Conditions**
  - Know which step filtering predicate is applied – earlier the better
- **Review Join Methods**
  - Nested Loops join: Usually efficient for smaller data sets
  - Hash Join: Useful on very large data sets (DW)
  - Merge Join: Efficient for larger data sets

# Examine the Explain Plan

- **First determine the type of query – OLTP or OLAP**
  - If OLTP, determine if indexes will help
    - Don’t use indexes on small tables (in memory) or when large dataset returned
  - Review join order, try for most selective join first
- **Look at rows removed & processed at each step**
  - Can an index reduce the number of rows accessed in the step?
- **If step is using an index, review the type of index**
  - Btree (can’t use ‘like’), Bitmap, Partial, Pattern Search, Covering, Compound, etc…
- **OLAP queries usually need most of the data for the result set**
  - Indexes shouldn’t be used
  - Consider semi-joins (exists) , anti-joins (not exists) & join order
  - Consider when Grouping occurs – Filter first, Group last or Group first, Join last
  - *Dombrovskaya, Henrietta; Bailliekova, Anna. PostgreSQL Query Optimization: The Ultimate Guide to Building Efficient Queries (p. 121). Apress. Kindle Edition.*

# Graphical Explain Plan

Foglight Cloud by Quest | Virtual Lab | Search

## Execution Plan

Download Plan

Zoom + 100% −

| 0.17 | 0.57 | 0.02 | 23.56 |
|---|---|---|---|
| execution time (ms) | planning time (ms) | slowest node (ms) | costliest node |

**AGGREGATE** <1 ms | 1 %
by i.inhparent
largest | bad estimate

**SORT** <1 ms | 14 %
by i.inhparent
slowest | largest | bad estimate

**HASH JOIN** <1 ms | 8 %
Right join
on (c.relfilenode = i.inhparent)
largest | bad estimate

**SEQ SCAN** <1 ms | 0 %
on pg_catalog.pg_class (c)
costliest | largest | bad estimate

**HASH** <1 ms | 2 %
largest | bad estimate

**SEQ SCAN** <1 ms | 2 %
on pg_catalog.pg_inherits (i)
largest | bad estimate

## SQL Text

Name
SELECT MIN(c.relname) as name, inhparent as parent, array_agg(i...

Query ID
5c99b7d4aea86e72412362a6092e58f2

SQL Text — Copy

```
SELECT
  MIN(c.relname) as name,
  inhparent as parent,
  array_agg(inhrelid) as children
FROM
  pg_catalog.pg_inherits i
LEFT JOIN
  pg_catalog.pg_class c
    on (
      i.inhparent=c.relfilenode
    )
GROUP BY
  inhparent
```

Explain Plan | Download SQL Text

# Several 3rd Party Tools for PostgreSQL Explain Plan

https://explain.dalibo.com/

explain.dalibo.com + New Plan Plan created on August 30th 2024, 8:25 pm

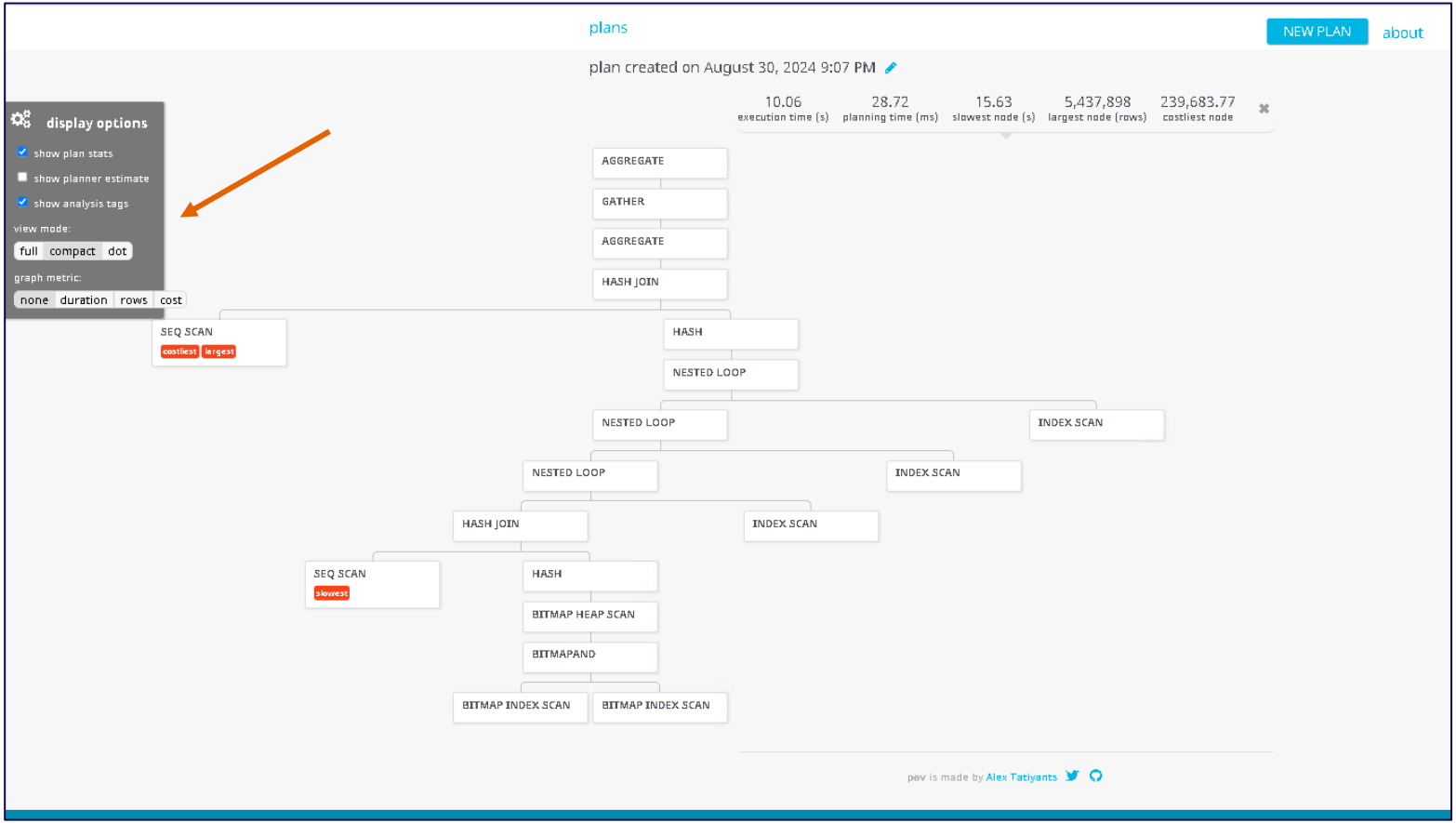
Plan Grid Raw Query Stats

Execution time: N/A Planning time: N/A Triggers: N/A

| | time | rows | estim | cost | loops | filter | |
|---|---|---|---|---|---|---|---|
| #1 | 0 | 1 | | 0.01 | | | Finalize Aggregate |
| #2 | 199 | 3 | 1.5 × ▾ | 1,000 | | | └ Gather |
| #3 | 0 | ~3 | | 3.86 | 3 | | └ Partial Aggregate |
| #4 | 1,046 | ~75 | 62 × ▲ | 17,900 | 3 | | └ Parallel Hash Semi Join on passenger.booking_id = bk_chi.booking_id |
| #5 | 4,450 | ~16,313,694 | 1.2 × ▲ | 240,000 | 3 | | ├ Parallel Seq Scan on passenger |
| #6 | 0 | ~27 | 19 × ▲ | 0 | 3 | | └ Parallel Hash |
| #7 | 0 | ~27 | 19 × ▲ | 1.75 | 3 | | └ Subquery Scan on bk_chi |
| #8 | 39 | ~27 | 19 × ▲ | 7,530 | 3 | | └ Parallel Hash Semi Join on booking.booking_id = booking_leg.booking_id |
| #9 | 573 | ~263,889 | 6.8 × ▲ | 13,900 | 3 | | ├ Parallel Hash Semi Join on booking.account_id = account.account_id |
| #10 | 1,261 | ~5,643,216 | 1.3 × ▲ | 112,000 | 3 | | │ ├ Parallel Seq Scan on booking |
| #11 | 6 | ~32,043 | 8.6 × ▲ | 0 | 3 | | │ └ Parallel Hash |
| #12 | 84 | ~32,043 | 8.6 × ▲ | 1,190 | 3 | | │ └ Parallel Hash Join on account.frequent_flyer_id = frequent_flyer.frequent_flyer_id |
| #13 | 193 | ~864,963 | 1.2 × ▲ | 11,900 | 3 | | │ ├ Parallel Seq Scan on account |
| #14 | 4 | ~32,043 | 1.8 × ▲ | 0 | 3 | | │ └ Parallel Hash |
| #15 | 14 | ~32,043 | 1.8 × ▲ | 2,750 | 3 | 75% | │ └ Parallel Seq Scan on frequent_flyer |
| #16 | 12 | ~774 | 1.5 × ▾ | 0 | 3 | | └ Parallel Hash |
| #17 | 257 | ~774 | 1.5 × ▾ | 3,160 | 3 | | └ Parallel Hash Join on booking_leg.flight_id = flight.flight_id |
| #18 | 3,024 | ~5,643,216 | 1.6 × ▾ | 225,000 | 3 | 68% | ├ Parallel Seq Scan on booking_leg |
| #19 | 0 | ~99 | 1.2 × ▲ | 0 | 3 | | └ Parallel Hash |
| #20 | 10 | 99 | 2.4 × ▾ | 8,770 | | 99% | └ Parallel Bitmap Heap Scan on flight |
| #21 | 1 | 18,668 | 1.1 × ▲ | 221 | | | └ Bitmap Index Scan using flight_departure_airport |

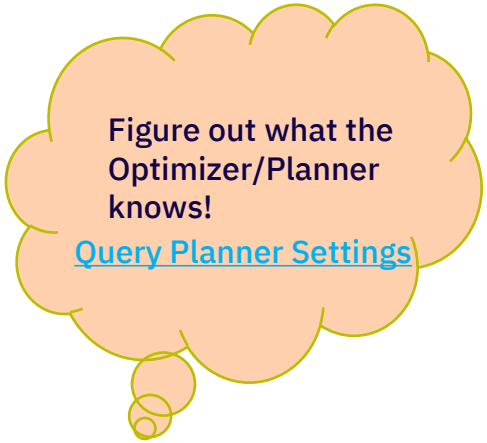
# Several 3rd Party Tools for PostgreSQL Explain Plan

https://pganalyze.com/index-advisor



# Gather Object Information

- **Understand objects in explain plans**
  - Table Definitions & Sizes
    - Is it a View?
      - Get underlying definition
    - Number of Rows / Partitioning?
  - Examine Columns in Where Clause
    - Know the Cardinality of columns
    - Is there Data Skew
      - Consider partial index
    - Are there indexes on the join / filtering columns
  - Index & Constraint Definitions
    - Entity Relationship Diagrams (ERDs) can help
- **Statistics Collection Configuration**
  - Analyze / Vacuum



# Case Study 1

## Who registered yesterday for SQL Tuning Class?

# Who registered yesterday for SQL Tuning

```
PREPARE billing (timestamp,timestamp) as
SELECT s.fname, s.lname, r.signup_date
FROM   test.student s
    INNER JOIN test.registration r ON s.student_id = r.student_id
    INNER JOIN test.class c ON r.class_id = c.class_id
WHERE  c.name  = 'SQL TUNING'
AND    r.signup_date BETWEEN $1 AND $2
AND    r.cancelled ='N';
```

Statement Summary

| | |
|---|---|
| Server | jgss2017.internal.cloudapp.net:5432 |
| Query ID | 3075482329797333000 |
| User | postgres |
| Database | test |
| Calls | 5.4 K |
| Avg Time | 45 ms |
| Total Time | 24.5 min |
| Avg Rows | 52.3 |
| Query | PREPARE billing (timestamp,timestamp) as SELECT s.fname, s.lname, r.signup_date FROM test.student s INNER JOIN test.registration r ON s.student_id = r.student_id INNER JOIN test.class c ON r.class_id = c.class_id WHERE c.name = $3 AND r.signup_date BETWEEN $1 AND $2 AND r.cancelled = $4 |

# Using Explain on Prepared Statements (version 16+)

Quest

- **Prepared Statements & PL/pgSQL functions use Cached Plans**
  - Postgres does not automatically cache plans for standard SQL statements

```
test=#
test=# explain (GENERIC_PLAN) SELECT s.fname, s.lname, r.signup_date
test-# FROM   test.student s
test-#     INNER JOIN test.registration r ON s.student_id = r.student_id
test-#     INNER JOIN test.class c ON r.class_id = c.class_id
test-# WHERE  c.name  = 'SQL TUNING'
test-# AND   r.signup_date BETWEEN $1 AND $2
test-# AND   r.cancelled ='N';
                                          QUERY PLAN
-----------------------------------------------------------------------------------------------
 Nested Loop  (cost=0.29..1950.45 rows=1 width=35)
   ->  Nested Loop  (cost=0.00..1948.11 rows=1 width=13)
         Join Filter: (r.class_id = c.class_id)
         ->  Seq Scan on registration r  (cost=0.00..1909.67 rows=398 width=18)
               Filter: ((signup_date >= $1) AND (signup_date <= $2) AND (cancelled = 'N'::bpchar))
         ->  Materialize  (cost=0.00..26.51 rows=2 width=5)
               ->  Seq Scan on class c  (cost=0.00..26.50 rows=2 width=5)
                     Filter: ((name)::text = 'SQL TUNING'::text)
   ->  Index Scan using pk_student on student s  (cost=0.29..2.33 rows=1 width=32)
         Index Cond: (student_id = r.student_id)
(10 rows)
```

# Explain (Analyze, Buffers)

```
explain (analyze, buffers)
SELECT s.fname, s.lname, r.signup_date
FROM   test.student s
    INNER JOIN test.registration r ON s.student_id = r.student_id
    INNER JOIN test.class c ON r.class_id = c.class_id
WHERE  c.name  = 'SQL TUNING'
AND   r.signup_date BETWEEN '2014-03-21 00:00:00' and '2014-03-28 00:00:00'
AND   r.cancelled ='N';

                                        QUERY PLAN
-----------------------------------------------------------------------------------------------------------------------
 Nested Loop  (cost=26.81..2045.81 rows=67 width=35) (actual time=1.181..32.482 rows=57 loops=1)
   Buffers: shared hit=695
   ->  Hash Join  (cost=26.52..2023.92 rows=67 width=13) (actual time=1.159..31.964 rows=57 loops=1)
         Hash Cond: (r.class_id = c.class_id)
         Buffers: shared hit=524
         ->  Seq Scan on registration r  (cost=0.00..1909.67 rows=33279 width=18) (actual time=0.030..24.081 rows=33259 loops=1)
               Filter: ((signup_date >= '2014-03-21 00:00:00'::timestamp without time zone)
                    AND (signup_date <= '2014-03-28 00:00:00'::timestamp without time zone)
                    AND (cancelled = 'N'::bpchar))
               Rows Removed by Filter: 46722
               Buffers: shared hit=510
         ->  Hash  (cost=26.50..26.50 rows=2 width=5) (actual time=0.256..0.257 rows=2 loops=1)
               Buckets: 1024  Batches: 1  Memory Usage: 9kB
               Buffers: shared hit=14
               ->  Seq Scan on class c  (cost=0.00..26.50 rows=2 width=5) (actual time=0.043..0.251 rows=2 loops=1)
                     Filter: ((name)::text = 'SQL TUNING'::text)
                     Rows Removed by Filter: 998
                     Buffers: shared hit=14
   ->  Index Scan using pk_student on student s  (cost=0.29..0.33 rows=1 width=32) (actual time=0.007..0.007 rows=1 loops=57)
         Index Cond: (student_id = r.student_id)
         Buffers: shared hit=171
 Planning:
   Buffers: shared hit=20
 Planning Time: 0.693 ms
 Execution Time: 32.553 ms
```

PKs / FKs only

33.246 ms

# Review Table & Indexes

```
test=# \d test.registration
                          Table "test.registration"
   Column     |             Type              | Collation | Nullable | Default
--------------+-------------------------------+-----------+----------+---------
 student_id   | numeric(18,0)                 |           |          |
 class_id     | numeric(18,0)                 |           |          |
 cancelled    | character(1)                  |           |          |
 signup_date  | timestamp without time zone   |           |          |
Indexes:
    "pk_registration" UNIQUE CONSTRAINT, btree (student_id, class_id, signup_date)
Foreign-key constraints:
    "registration_class_id_fkey" FOREIGN KEY (class_id) REFERENCES test.class(class_id)
    "registration_student_id_fkey" FOREIGN KEY (student_id) REFERENCES test.student(student_id)
```

| # of Rows |
|---|
| class 1,000 |
| student 10,000 |
| registration 79,981 |

```
test=# SELECT tablename, indexname FROM pg_indexes WHERE tablename in ('class','student','registration');
  tablename   |     indexname
--------------+----------------
 student      | pk_student
 class        | pk_class
 registration | pk_registration
(3 rows)
```

```
test=# \di+ test.*
                                                  List of relations
 Schema |      Name       | Type  |  Owner   |     Table      | Persistence | Access method |  Size   |
--------+-----------------+-------+----------+----------------+-------------+---------------+---------+
 test   | pk_class        | index | postgres | class          | permanent   | btree         | 56 kB   |
 test   | pk_registration | index | postgres | registration   | permanent   | btree         | 4248 kB |
 test   | pk_student      | index | postgres | student        | permanent   | btree         | 304 kB  |
(3 rows)
```

# Find the Driving table

- **Need to know the size of the actual data sets in each step**
  - In Joins (Right, Left, Outer)
  - What are the filtering predicates
  - When is each filtering predicate applied
    - Try to filter earlier rather than later
- **Compare size of final result set with # of rows at each step**
- **Find the driving table**
    - To reduce buffers (I/O)

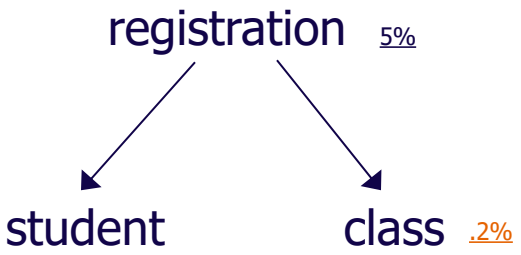
```
SELECT s.fname, s.lname, r.signup_date
FROM   student s
   INNER JOIN registration r ON s.student_id = r.student_id
   INNER JOIN class c ON r.class_id = c.class_id
WHERE  c.name  = 'SQL TUNING'
AND    r.signup_date BETWEEN  $1 AND $2
AND    r.cancelled = 'N'
```

**Joins**

**Filtering Predicates**

# SQL Diagramming

- **Great Book “SQL Tuning” by Dan Tow**
  - Oldie but a goodie that teaches SQL Diagramming
  - http://www.singingsql.com




```
select count(1) from registration where cancelled = 'N'
and signup_date between '2022-12-10 00:00' and '2022-12-11 00:00'

4344 / 79,981 * 100  = 5.43%
                                    5.43

select count(1) from class where name = 'SQL TUNING'

    2 / 1000 * 100 = .2
```

# Drive the Query with Class

- **CREATE INDEX cl_name ON test.class(name);**

```
                                        QUERY PLAN
-----------------------------------------------------------------------------------------------------------------------
Nested Loop  (cost=10.36..2029.36 rows=67 width=35) (actual time=1.117..31.488 rows=57 loops=1)
  Buffers: shared hit=685
  ->  Hash Join  (cost=10.07..2007.46 rows=67 width=13) (actual time=1.096..31.019 rows=57 loops=1)
        Hash Cond: (r.class_id = c.class_id)
        Buffers: shared hit=514
        ->  Seq Scan on registration r  (cost=0.00..1909.67 rows=33279 width=18) (actual time=0.017..23.553 rows=33259 loops=1)
              Filter: ((signup_date >= '2014-03-21 00:00:00'::timestamp without time zone)
                   AND (signup_date <= '2014-03-28 00:00:00'::timestamp without time zone)
                   AND (cancelled = 'N'::bpchar))
              Rows Removed by Filter: 46722
              Buffers: shared hit=510
        ->  Hash  (cost=10.05..10.05 rows=2 width=5) (actual time=0.069..0.070 rows=2 loops=1)
              Buckets: 1024  Batches: 1  Memory Usage: 9kB
              Buffers: shared hit=4
              ->  Bitmap Heap Scan on class c  (cost=4.29..10.05 rows=2 width=5) (actual time=0.060..0.064 rows=2 loops=1)
                    Recheck Cond: ((name)::text = 'SQL TUNING'::text)
                    Heap Blocks: exact=2
                    Buffers: shared hit=4
                    ->  Bitmap Index Scan on cl_name  (cost=0.00..4.29 rows=2 width=0) (actual time=0.054..0.054 rows=2 loops=1)
                          Index Cond: ((name)::text = 'SQL TUNING'::text)
                          Buffers: shared hit=2
  ->  Index Scan using pk_student on student s  (cost=0.29..0.33 rows=1 width=32) (actual time=0.007..0.007 rows=1 loops=57)
        Index Cond: (student_id = r.student_id)
        Buffers: shared hit=171
Planning:
  Buffers: shared hit=50
Planning Time: 5.034 ms
Execution Time: 31.594 ms
```

36.628 ms

Previous

33.246 ms

# Why Seq Scan on Registration?

- **Can't use Primary Key as class_id is not left leading column**

```
test=# \d test.registration
                                Table "test.registration"
   Column    |            Type             | Collation | Nullable |      Default
-------------+-----------------------------+-----------+----------+-------------------
 student_id  | numeric(18,0)               |           |          |
 class_id    | numeric(18,0)               |           |          |
 cancelled   | character(1)                |           |          |
 signup_date | timestamp without time zone |           |          | CURRENT_TIMESTAMP
Indexes:
    "pk_registration" UNIQUE CONSTRAINT, btree (student_id, class_id, signup_date)
Foreign-key constraints:
    "registration_class_id_fkey" FOREIGN KEY (class_id) REFERENCES test.class(class_id)
    "registration_student_id_fkey" FOREIGN KEY (student_id) REFERENCES test.student(student_id)
```

- **Not much difference in throughput – 6k vs 5.4k (685 vs 695 buffers)**
  - Needs more information to drive by Class

# Add Index on Registration (Class_id)

- **create index REG_ALT on test.registration(class_id);**

```
                              QUERY PLAN
-----------------------------------------------------------------------------------------------------------------------
Nested Loop  (cost=9.60..450.25 rows=67 width=35) (actual time=0.107..0.750 rows=57 loops=1)
  Buffers: shared hit=328
  ->  Nested Loop  (cost=9.32..428.36 rows=67 width=13) (actual time=0.093..0.389 rows=57 loops=1)
        Buffers: shared hit=157
        ->  Bitmap Heap Scan on class c  (cost=4.29..10.05 rows=2 width=5) (actual time=0.034..0.036 rows=2 loops=1)
              Recheck Cond: ((name)::text = 'SQL TUNING'::text)
              Heap Blocks: exact=2
              Buffers: shared hit=4
              ->  Bitmap Index Scan on cl_name  (cost=0.00..4.29 rows=2 width=0) (actual time=0.028..0.028 rows=2 loops=1)
                    Index Cond: ((name)::text = 'SQL TUNING'::text)
                    Buffers: shared hit=2
        ->  Bitmap Heap Scan on registration r  (cost=5.03..208.83 rows=33 width=18) (actual time=0.044..0.163 rows=29 loops=2)
              Recheck Cond: (class_id = c.class_id)
              Filter: ((signup_date >= '2014-03-21 00:00:00'::timestamp without time zone)
                  AND (signup_date <= '2014-03-28 00:00:00'::timestamp without time zone)
                  AND (cancelled = 'N'::bpchar))
              Rows Removed by Filter: 47
              Heap Blocks: exact=146
              Buffers: shared hit=153
              ->  Bitmap Index Scan on reg_alt  (cost=0.00..5.02 rows=80 width=0) (actual time=0.027..0.027 rows=76 loops=2)
                    Index Cond: (class_id = c.class_id)
                    Buffers: shared hit=7
  ->  Index Scan using pk_student on student s  (cost=0.29..0.33 rows=1 width=32) (actual time=0.005..0.005 rows=1 loops=57)
        Index Cond: (student_id = r.student_id)
        Buffers: shared hit=171
Planning:
  Buffers: shared hit=46
Planning Time: 3.223 ms
Execution Time: 0.833 ms
```

4.056 ms

Previous>

36.628 ms

33.246 ms

# Add Covering Index on Registration

- **create index REG_ALT on test.registration(class_id, student_id, signup_date) include (cancelled);**

```
                                QUERY PLAN
-----------------------------------------------------------------------------------------------------------------------
 Nested Loop  (cost=4.99..44.26 rows=67 width=35) (actual time=0.099..0.455 rows=57 loops=1)
   Buffers: shared hit=183
   ->  Nested Loop  (cost=4.71..22.37 rows=67 width=13) (actual time=0.084..0.139 rows=57 loops=1)
         Buffers: shared hit=12
         ->  Bitmap Heap Scan on class c  (cost=4.29..10.05 rows=2 width=5) (actual time=0.035..0.037 rows=2 loops=1)
               Recheck Cond: ((name)::text = 'SQL TUNING'::text)
               Heap Blocks: exact=2
               Buffers: shared hit=4
               ->  Bitmap Index Scan on cl_name  (cost=0.00..4.29 rows=2 width=0) (actual time=0.028..0.028 rows=2 loops=1)
                     Index Cond: ((name)::text = 'SQL TUNING'::text)
                     Buffers: shared hit=2
         ->  Index Only Scan using reg_alt on registration r  (cost=0.42..5.83 rows=33 width=18) (actual time=0.030..0.045 rows=29 loops=2)
               Index Cond: ((class_id = c.class_id)
                         AND (signup_date >= '2014-03-21 00:00:00'::timestamp without time zone)
                         AND (signup_date <= '2014-03-28 00:00:00'::timestamp without time zone))
               Filter: (cancelled = 'N'::bpchar)
               Heap Fetches: 0
               Buffers: shared hit=8
   ->  Index Scan using pk_student on student s  (cost=0.29..0.33 rows=1 width=32) (actual time=0.005..0.005 rows=1 loops=57)
         Index Cond: (student_id = r.student_id)
         Buffers: shared hit=171
 Planning:
   Buffers: shared hit=28
 Planning Time: 0.862 ms
 Execution Time: 0.533 ms
```

1.395 ms

Previous >

4.056 ms

36.628 ms

33.246 ms

Shared Hits

IX 1: 695

IX 2: 328

1X 3: 183

# Engineer out the Stupid

Other_bad_things_about_postgresql

- **Selecting unnecessary columns or using wildcards (*) in Select clause**
  - Adding extra columns can slow down query performance
  - Increase the amount of memory (I/O) & CPU needed
- **Using ambiguous table aliases**
  - Using ambiguous table aliases leads to confusion
  - Can cause errors when writing complex queries
  - Use clear, meaning full aliases
- **Not filtering data - not using a WHERE clause**
  - Always include a WHERE clause in your query
  - Databases are best at filtering data so don't forward that job to client
    - Reduces Network Latency
- **Not using appropriate JOINs**
  - Using  inefficient JOINs in a query can lead to performance issues
  - Nested Loops are good for large/small table lookups, Merge & Hash joins for large/large tables

More stupid things in Appendix

# Case Study 2

# Flights by City & Day of Week

# SQL Taking the Most Time

Foglight by Quest

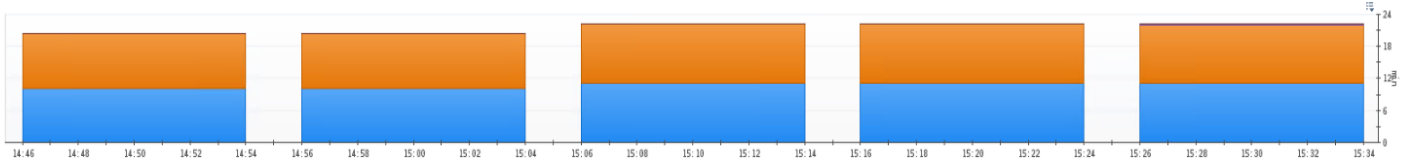
Search

Databases > PostgreSQL Server: jgss2017.internal.cloudapp.net:5432 > Statements for Postgresqlagent

Tuesday, Apr 2, 2024 12:17 PM - Now 4 hours | Reports

Overview | Databases | Tables | Connections | BG Writer | Locks | Configuration | **Statements** | Tablespaces

Order By: Total Time 10 Apply




total_time
- SELECT pg_locks.pid as pid, locktype, virtualtrans...
- SELECT funcid, schemaname, funcname, calls, total_...
- CALL public.get_pop_flight('Little Rock, AR','2015...
- SELECT s.queryid as queryid, s.userid as userid, s...
- SELECT name, setting, unit from pg_settings
- SELECT d.datid as dbid, d.datname as name, t.spcna...
- PREPARE billing (timestamp,timestamp) asSELECT s....
- SELECT t.oid as oid, spcname as name, u.usename as...
- SELECT o.carrier, uc.description AS carrier_name, ...
- fetch all in "pop_flights"

Workload

There Is No Data To Display

Total Statement Counts

CPU Utilization

There Is No Data To Display

Logical Disk Reads Rate

There Is No Data To Display

Physical Disks Bytes Read Rate

There Is No Data To Display

Search

| | | Statement | | | | | Memory Blocks | | | | Shared Blocks |
|---|---|---|---|---|---|---|---|---|---|---|---|
| User | DB | Query | Calls | Average | Total | Avg Rows | Write Time | Read Time | Read Time per Query | Read Time per Row | Hit Pct |
| postgres | test | PREPARE billing (timestamp,timestamp) as SELECT s.fname, s.lname, ... | 92.5 K | 0.40 ms | 37.3 sec | 52.3 | 0 ms | 0 ms | 0.00 ms | 0.00 ms | 100.00 % |
| postgres | dot | fetch all in "pop_flights" | 1.7 K | 3.69 s | 1.7 hr | 205.8 | 0 ms | 0 ms | 0.00 ms | 0.00 ms | 39.78 % |
| postgres | dot | SELECT o.carrier, uc.description AS carrier_name, o.fl_date,o.fl_num... | 1.7 K | 3.68 s | 1.7 hr | 203.5 | 0 ms | 0 ms | 0.00 ms | 0.00 ms | 41.35 % |
| postgres | dot | CALL public.get_pop_flight('Little Rock, AR','2015-02-01','2015-02-16',... | 213 | 0.10 ms | 22.1 ms | 0 | 0 ms | 0 ms | 0.00 ms | 0.00 ms | n/a |
| postgres | dot | CALL public.get_pop_flight('Little Rock, AR','2015-02-01','2015-02-16',... | 213 | 0.13 ms | 28.6 ms | 0 | 0 ms | 0 ms | 0.00 ms | 0.00 ms | n/a |
| postgres | dot | CALL public.get_pop_flight('Little Rock, AR','2015-02-01','2015-02-16',... | 213 | 0.13 ms | 28.7 ms | 0 | 0 ms | 0 ms | 0.00 ms | 0.00 ms | n/a |
| postgres | dot | CALL public.get_pop_flight('Little Rock, AR','2015-02-01','2015-02-16',... | 213 | 0.14 ms | 28.9 ms | 0 | 0 ms | 0 ms | 0.00 ms | 0.00 ms | n/a |

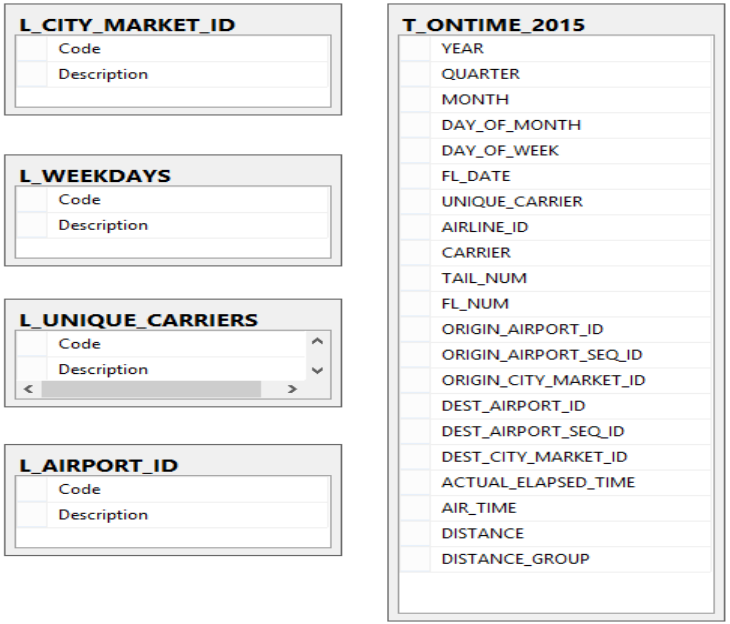
# Flights by City & Day of Week

```
CREATE OR REPLACE PROCEDURE
      get_pop_flight(_city varchar,_beg_date date, _end_date date,_day_of_week varchar, INOUT pop_flights refcursor)
LANGUAGE 'plpgsql'
AS $BODY$
BEGIN
OPEN pop_flights FOR SELECT o.carrier, uc.description AS carrier_name, o.fl_date,o.fl_num,o.tail_num
                              ,ao.description AS origin_airport,co.Description AS origin_city ,ad.description AS destination_airport
                              ,cd.Description AS destination_city ,w.Description Day_of_Wed
FROM public.t_ontime o
INNER JOIN L_UNIQUE_CARRIERS AS uc ON uc.Code = o.UNIQUE_CARRIER
        INNER JOIN L_AIRPORT_ID AS ao ON ao.Code = o.ORIGIN_AIRPORT_ID
        INNER JOIN L_AIRPORT_ID AS ad ON ad.Code = o.DEST_AIRPORT_ID
        INNER JOIN L_CITY_MARKET_ID AS co ON co.Code = o.ORIGIN_CITY_MARKET_ID
        INNER JOIN L_CITY_MARKET_ID AS cd ON cd.Code = o.DEST_CITY_MARKET_ID
        INNER JOIN L_WEEKDAYS AS w ON w.Code = o.DAY_OF_WEEK
where fl_date BETWEEN _beg_date AND _end_date
        AND co.Description = _city
        AND w.Description = _day_of_week;
END;
$BODY$;
```

```
BEGIN;
CALL public.get_pop_flight('Little Rock, AR','2015-02-01','2015-0 16','Sunday','pop_flights');
fetch all in "pop_flights";
COMMIT;
```

# Star Schema

- [US DOT - On-time Performance]



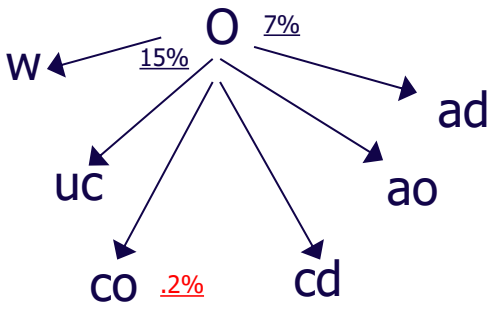

| | |
|---|---|
| L_UNIQUE_CARRIERS: | 1620 |
| L_AIRPORT_ID: | 6438 |
| L_CITY_MARKET_ID: | 5823 |
| L_WEEKDAYS: | 8 |
| T_ONTIME: | 6784044 |

# Examine the Explain Plan

2.07 seconds

```
                                                    QUERY PLAN
-----------------------------------------------------------------------------------------------------------------------------------
Nested Loop  (cost=1016.79..159139.35 rows=1 width=263) (actual time=769.880..2069.920 rows=62 loops=1)
  Join Filter: (o.dest_city_market_id = cd.code)
  Rows Removed by Join Filter: 360964
  Buffers: shared hit=21617 read=103930
  ->  Nested Loop  (cost=1016.79..158970.33 rows=1 width=250) (actual time=769.617..1959.944 rows=62 loops=1)
        Join Filter: (o.dest_airport_id = ad.code)
        Rows Removed by Join Filter: 399094
        Buffers: shared hit=19261 read=103930
        ->  Nested Loop  (cost=1016.79..158767.48 rows=1 width=217) (actual time=769.282..1829.759 rows=62 loops=1)
              Join Filter: (o.origin_airport_id = ao.code)
              Rows Removed by Join Filter: 399094
              Buffers: shared hit=15665 read=103930
              ->  Nested Loop  (cost=1016.79..158564.62 rows=1 width=184) (actual time=768.353..1705.455 rows=62 loops=1)
                    Join Filter: ((o.unique_carrier)::bpchar = uc.code)
                    Rows Removed by Join Filter: 100378
                    Buffers: shared hit=12069 read=103930
                    ->  Nested Loop  (cost=1016.79..158515.17 rows=1 width=167) (actual time=768.195..1664.851 rows=62 loops=1)
                          Join Filter: (o.origin_city_market_id = co.code)
                          Rows Removed by Join Filter: 32132
                          Buffers: shared hit=11263 read=103930
                          ->  Seq Scan on l_city_market_id co  (cost=0.00..110.79 rows=1 width=21) (actual time=0.448..1.428 rows=1 loops=1)
                                Filter: ((description)::text = 'Little Rock, AR'::text)
                                Rows Removed by Filter: 5822
                                Buffers: shared hit=38
                          ->  Gather  (cost=1016.79..158360.67 rows=3497 width=154) (actual time=684.680..1615.868 rows=32194 loops=1)
                                Workers Planned: 2
                                Workers Launched: 2
                                Buffers: shared hit=11225 read=103930
                                ->  Hash Join  (cost=16.79..157010.97 rows=1457 width=154) (actual time=559.696..1481.533 rows=10731 loops=3)
                                      Hash Cond: (o.day_of_week = w.code)
                                      Buffers: shared hit=11225 read=103930
                                      ->  Parallel Seq Scan on t_ontime o
                                            (cost=0.00..156615.38 rows=97130 width=40) (actual time=558.148..1460.402 rows=76606 loops=3)
                                            Filter: ((fl_date >= '2015-02-01'::date) AND (fl_date <= '2015-02-16'::date))
                                            Rows Removed by Filter: 2145399
                                            Buffers: shared hit=11030 read=103930
                                      ->  Hash  (cost=16.75..16.75 rows=3 width=122) (actual time=0.966..0.968 rows=1 loops=3)
                                            Buckets: 1024  Batches: 1  Memory Usage: 9kB
                                            Buffers: shared hit=3
                                            ->  Seq Scan on l_weekdays w
                                                  (cost=0.00..16.75 rows=3 width=122) (actual time=0.934..0.936 rows=1 loops=3)
                                                  Filter: ((description)::text = 'Friday'::text)
                                                  Rows Removed by Filter: 7
                                                  Buffers: shared hit=3
                    ->  Seq Scan on l_unique_carriers uc  (cost=0.00..29.20 rows=1620 width=31) (actual time=0.004..0.237 rows=1620 loops=62)
                          Buffers: shared hit=806
              ->  Seq Scan on l_airport_id ao  (cost=0.00..122.38 rows=6438 width=41) (actual time=0.003..0.838 rows=6438 loops=62)
                    Buffers: shared hit=3596
        ->  Seq Scan on l_airport_id ad  (cost=0.00..122.38 rows=6438 width=41) (actual time=0.003..0.836 rows=6438 loops=62)
              Buffers: shared hit=3596
  ->  Seq Scan on l_city_market_id cd  (cost=0.00..96.23 rows=5823 width=21) (actual time=0.004..0.748 rows=5823 loops=62)
        Buffers: shared hit=2356
Planning Time: 2.060 ms
Execution Time: 2070.099 ms
```

# Find the Driving Table




Filtering Selectivity

```
select count(1) from t_ontime where  origin_city_market_id = 32600;
select 12953.000/6666016.000 * 100;  = .194   or .2%

Select count(1) from t_ontime where fl_date
                        between '2015-12-01 00:00:00.000' and'2015-12-31 00:00:00.000‘;
select 469717.00 / 6666016.00 * 100; = 7.04 or 7%

select count(1) from t_ontime where day_of_week = 1;
select 987647.00/6666016.00 * 100; = 14.81 or 15%
```

# Tune the Query

- **Create index on T_ONTIME & L_CITY_MARKET_ID**
  - create index OCO_MARKET_DESC on public.T_ONTIME(origin_city_market);
  - create index CO_MARKET_DESC on public.L_CITY_MARKET_ID(description);
  - create unique index PK_CITY_MARKET on public.L_CITY_MARKET_ID(code);

```
dot=# \d public.t_ontime
                          Table "public.t_ontime"
        Column          |          Type          | Collation |
------------------------+------------------------+-----------+
 year                   | integer                |           |
 quarter                | integer                |           |
 month                  | integer                |           |
 day_of_month           | integer                |           |
 day_of_week            | integer                |           |
 fl_date                | date                   |           |
 unique_carrier         | character varying(10)  |           |
 airline_id             | integer                |           |
 carrier                | character varying(10)  |           |
 tail_num               | character varying(10)  |           |
 fl_num                 | integer                |           |
 origin_airport_id      | integer                |           |
 origin_airport_seq_id  | integer                |           |
 origin_city_market_id  | integer                |           |
 dest_airport_id        | integer                |           |
 dest_airport_seq_id    | integer                |           |
 dest_city_market_id    | integer                |           |
 actual_elapsed_time    | numeric(6,2)           |           |
 air_time               | numeric(6,2)           |           |
 flights                | numeric(6,2)           |           |
 distance               | numeric(6,2)           |           |
 distance_group         | numeric                |           |
 total_add_gtime        | numeric                |           |
Indexes:
    "oco_market_desc" btree (origin_city_market_id)
```

```
dot=# \d public.l_city_market_id
                      Table "public.l_city_market_id"
   Column    |          Type          | Collation | Nullable |
-------------+------------------------+-----------+----------+
 code        | integer                |           |          |
 description | character varying(100) |           |          |
Indexes:
    "co_market_desc" btree (description)
    "pk_city_market" UNIQUE, btree (code)
```

```
QUERY PLAN
Nested Loop  (cost=254.79..58285.68 rows=1 width=263) (actual time=3.836..321.645 rows=62 loops=1)
  Buffers: shared hit=8754 read=2787
  ->  Nested Loop  (cost=254.51..58285.37 rows=1 width=250) (actual time=3.795..320.654 rows=62 loops=1)
        Join Filter: (o.dest_airport_id = ad.code)
        Rows Removed by Join Filter: 399094
        Buffers: shared hit=8575 read=2780
        ->  Nested Loop  (cost=254.51..58082.52 rows=1 width=217) (actual time=2.872..212.043 rows=62 loops=1)
              Join Filter: (o.origin_airport_id = ao.code)
              Rows Removed by Join Filter: 399094
              Buffers: shared hit=4979 read=2780
              ->  Nested Loop  (cost=254.51..57879.66 rows=1 width=184) (actual time=2.129..101.717 rows=62 loops=1)
                    Join Filter: ((o.unique_carrier)::bpchar = uc.code)
                    Rows Removed by Join Filter: 100378
                    Buffers: shared hit=1383 read=2780
                    ->  Nested Loop  (cost=254.51..57830.21 rows=1 width=167) (actual time=2.016..65.792 rows=62 loops=1)
                          Join Filter: (o.day_of_week = w.code)
                          Rows Removed by Join Filter: 358
                          Buffers: shared hit=577 read=2780
                          ->  Nested Loop  (cost=254.51..57811.66 rows=40 width=53) (actual time=1.964..65.327 rows=420 loops=1)
                                Buffers: shared hit=577 read=2779
                                ->  Index Scan using co_market_desc on l_city_market_id co  (cost=0.28..8.30 rows=1 width=21) (actual time=0.063..0.066 rows=1 loops=1)
                                      Index Cond: ((description)::text = 'Little Rock, AR'::text)
                                      Buffers: shared hit=1 read=2
                                ->  Bitmap Heap Scan on t_ontime o  (cost=254.23..57795.26 rows=810 width=40) (actual time=1.891..65.043 rows=420 loops=1)
                                      Recheck Cond: (origin_city_market_id = co.code)
                                      Filter: ((fl_date >= '2015-02-01'::date) AND (fl_date <= '2015-02-16'::date))
                                      Rows Removed by Filter: 12533
                                      Heap Blocks: exact=3339
                                      Buffers: shared hit=576 read=2777
                                      ->  Bitmap Index Scan on oco_market_desc  (cost=0.00..254.03 rows=23146 width=0) (actual time=1.243..1.244 rows=12953 loops=1)
                                            Index Cond: (origin_city_market_id = co.code)
                                            Buffers: shared hit=1 read=13
                          ->  Materialize  (cost=0.00..16.77 rows=3 width=122) (actual time=0.000..0.000 rows=1 loops=420)
                                Buffers: shared read=1
                                ->  Seq Scan on l_weekdays w  (cost=0.00..16.75 rows=3 width=122) (actual time=0.026..0.027 rows=1 loops=1)
                                      Filter: ((description)::text = 'Friday'::text)
                                      Rows Removed by Filter: 7
                                      Buffers: shared read=1
                    ->  Seq Scan on l_unique_carriers uc  (cost=0.00..29.20 rows=1620 width=31) (actual time=0.003..0.224 rows=1620 loops=62)
                          Buffers: shared hit=806
              ->  Seq Scan on l_airport_id ao  (cost=0.00..122.38 rows=6438 width=41) (actual time=0.002..0.768 rows=6438 loops=62)
                    Buffers: shared hit=3596
        ->  Seq Scan on l_airport_id ad  (cost=0.00..122.38 rows=6438 width=41) (actual time=0.003..0.790 rows=6438 loops=62)
              Buffers: shared hit=3596
  ->  Index Scan using pk_city_market on l_city_market_id cd  (cost=0.28..0.30 rows=1 width=21) (actual time=0.011..0.011 rows=1 loops=62)
        Index Cond: (code = o.dest_city_market_id)
        Buffers: shared hit=179 read=7
Planning:
  Buffers: shared hit=5 read=15
Planning Time: 4.337 ms
Execution Time: 321.892 ms
```

326.229 ms

Previous> 2.07 S

# Performance Improved?

Databases > ... > Statements for Postgresqlagent > Statement Details

Tuesday, Apr 2, 2024 04:39:03 PM - Now 1 hour | Reports

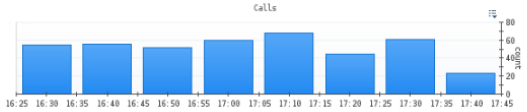
Overview | Databases | Tables | Connections | BG Writer | Locks | Configuration | **Statements** | Tablespaces

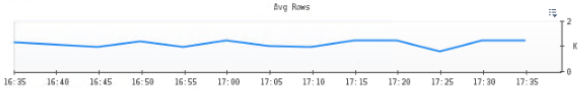
Statement Summary

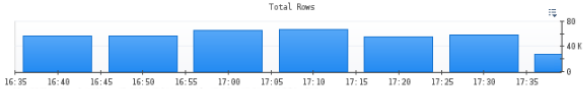
| | |
|---|---|
| Agent | Postgresqlagent |
| Server | jgss2017.internal.cloudapp.net:5432 |
| Query ID | -1454855392704078340 |
| User | postgres |
| Database | dot |
| Calls | 28 |
| Avg Time | 4,843 ms |
| Total Time | 29.4 min |
| Avg Rows | 1,062.7 |
| | SELECT o.carrier, uc.description AS carrier_name, o.fl_date,o.fl_num,o.tail_num ,ao.description AS origin_airport,co.Description AS origin_city ,ad.description AS destination_airport ,cd.Description AS destination_city ,w.Description Day_of_Wed from public.t_ontime o INNER JOIN L_UNIQUE_CARRIERS AS uc ON uc.Code = |

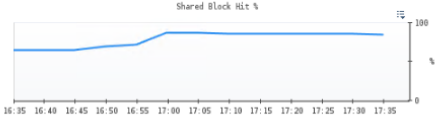
Statement on other Databases/Servers

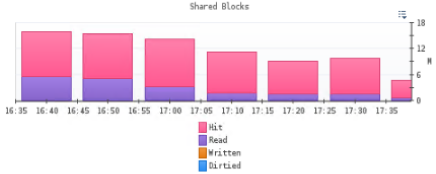
| Statement | | | | | | Memory Blocks | | | |
|---|---|---|---|---|---|---|---|---|---|
| Name | DB | Calls | Average | Total | Avg Rows | Write Time | Read Time | Read Time per Query | Read Time per Row |
| There Is No Data To Display | | | | | | | | | |

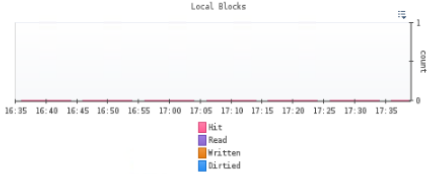


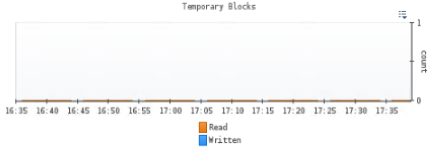





Explain Plan (PostgreSQL Administrator use only)

# Adjust the index

- **Create index OCO_MARKET_DESC_FL_DATE on public.T_ONTIME(origin_city_market_id, fl_date);**

```
dot=# \d public.t_ontime
                          Table "public.t_ontime"
        Column          |         Type          | Collation | Nullable |
------------------------+-----------------------+-----------+----------+
 year                   | integer               |           |          |
 quarter                | integer               |           |          |
 month                  | integer               |           |          |
 day_of_month           | integer               |           |          |
 day_of_week            | integer               |           |          |
 fl_date                | date                  |           |          |
 unique_carrier         | character varying(10) |           |          |
 airline_id             | integer               |           |          |
 carrier                | character varying(10) |           |          |
 tail_num               | character varying(10) |           |          |
 fl_num                 | integer               |           |          |
 origin_airport_id      | integer               |           |          |
 origin_airport_seq_id  | integer               |           |          |
 origin_city_market_id  | integer               |           |          |
 dest_airport_id        | integer               |           |          |
 dest_airport_seq_id    | integer               |           |          |
 dest_city_market_id    | integer               |           |          |
 actual_elapsed_time    | numeric(6,2)          |           |          |
 air_time               | numeric(6,2)          |           |          |
 flights                | numeric(6,2)          |           |          |
 distance               | numeric(6,2)          |           |          |
 distance_group         | numeric               |           |          |
 total_add_gtime        | numeric               |           |          |
Indexes:
    "oco_market_desc_fl_date" btree (origin_city_market_id, fl_date)
```

```
QUERY PLAN
Nested Loop  (cost=3239.51..3435.51 rows=1 width=263) (actual time=2.879..36.186 rows=62 loops=1)
  Buffers: shared hit=1236
  ->  Nested Loop  (cost=3239.23..3435.21 rows=1 width=250) (actual time=2.861..35.865 rows=62 loops=1)
        Join Filter: ((o.unique_carrier)::bpchar = uc.code)
        Rows Removed by Join Filter: 100378
        Buffers: shared hit=1050
        ->  Hash Join  (cost=3239.23..3385.76 rows=1 width=233) (actual time=2.518..4.186 rows=62 loops=1)
              Hash Cond: (ad.code = o.dest_airport_id)
              Buffers: shared hit=244
              ->  Seq Scan on l_airport_id ad  (cost=0.00..122.38 rows=6438 width=41) (actual time=0.014..0.667 rows=6438 loops=1)
                    Buffers: shared hit=58
              ->  Hash  (cost=3239.22..3239.22 rows=1 width=200) (actual time=2.470..2.473 rows=62 loops=1)
                    Buckets: 1024  Batches: 1  Memory Usage: 17kB
                    Buffers: shared hit=186
                    ->  Hash Join  (cost=3092.69..3239.22 rows=1 width=200) (actual time=1.587..2.443 rows=62 loops=1)
                          Hash Cond: (ao.code = o.origin_airport_id)
                          Buffers: shared hit=186
                          ->  Seq Scan on l_airport_id ao  (cost=0.00..122.38 rows=6438 width=41) (actual time=0.006..0.648 rows=6438 loops=1)
                                Buffers: shared hit=58
                          ->  Hash  (cost=3092.67..3092.67 rows=1 width=167) (actual time=0.799..0.802 rows=62 loops=1)
                                Buckets: 1024  Batches: 1  Memory Usage: 14kB
                                Buffers: shared hit=128
                                ->  Nested Loop  (cost=15.04..3092.67 rows=1 width=167) (actual time=0.133..0.765 rows=62 loops=1)
                                      Join Filter: (o.day_of_week = w.code)
                                      Rows Removed by Join Filter: 358
                                      Buffers: shared hit=128
                                      ->  Nested Loop  (cost=15.04..3074.12 rows=40 width=53) (actual time=0.107..0.531 rows=420 loops=1)
                                            Buffers: shared hit=127
                                            ->  Index Scan using co_market_desc on l_city_market_id co  (cost=0.28..8.30 rows=1 width=21) (actual time=0.039..0.040 rows=1 loops=1)
                                                  Index Cond: ((description)::text = 'Little Rock, AR'::text)
                                                  Buffers: shared hit=3
                                            ->  Bitmap Heap Scan on t_ontime o  (cost=14.76..3057.72 rows=810 width=40) (actual time=0.060..0.304 rows=420 loops=1)
                                                  Recheck Cond: ((origin_city_market_id = co.code) AND (fl_date >= '2015-02-01'::date) AND (fl_date <= '2015-02-16'::date))
                                                  Heap Blocks: exact=120
                                                  Buffers: shared hit=124
                                                  ->  Bitmap Index Scan on oco_market_desc_fl_date  (cost=0.00..14.56 rows=810 width=0) (actual time=0.038..0.038 rows=420 loops=1)
                                                        Index Cond: ((origin_city_market_id = co.code) AND (fl_date >= '2015-02-01'::date) AND (fl_date <= '2015-02-16'::date))
                                                        Buffers: shared hit=4
                                      ->  Materialize  (cost=0.00..16.77 rows=3 width=122) (actual time=0.000..0.000 rows=1 loops=420)
                                            Buffers: shared hit=1
                                            ->  Seq Scan on l_weekdays w  (cost=0.00..16.75 rows=3 width=122) (actual time=0.011..0.012 rows=1 loops=1)
                                                  Filter: ((description)::text = 'Friday'::text)
                                                  Rows Removed by Filter: 7
                                                  Buffers: shared hit=1
        ->  Seq Scan on l_unique_carriers uc  (cost=0.00..29.20 rows=1620 width=31) (actual time=0.002..0.195 rows=1620 loops=62)
              Buffers: shared hit=806
  ->  Index Scan using pk_city_market on l_city_market_id cd  (cost=0.28..0.30 rows=1 width=21) (actual time=0.004..0.004 rows=1 loops=62)
        Index Cond: (code = o.dest_city_market_id)
        Buffers: shared hit=186
Planning:
  Buffers: shared hit=28
Planning Time: 4.669 ms
Execution Time: 36.378 ms
```

41.047 ms

Previous>

326.229 ms

2.07 S

# Engineer out the Stupid

- **No Primary or Foreign Keys!** (See appendix for more Stupid Things)

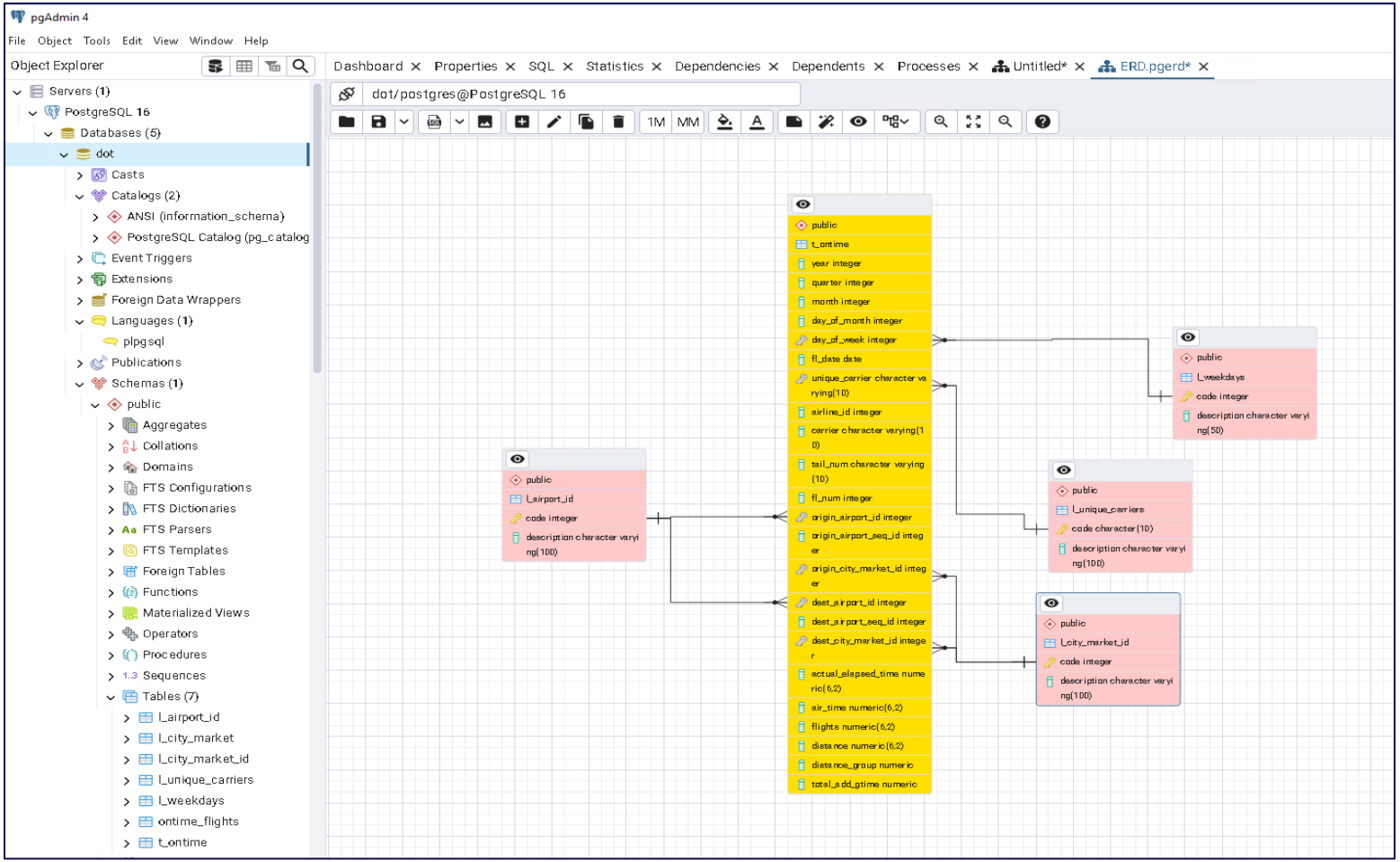
```
dot-# \di+ public.*
                                                    List of relations
 Schema |          Name            | Type  |  Owner   |       Table        | Persistence | Access method |  Size  |
--------+--------------------------+-------+----------+--------------------+-------------+---------------+--------+
 public | oco_market_desc_fl_date  | index | postgres | t_ontime           | permanent   | btree         | 47 MB  |
 public | pk_city_market           | index | postgres | l_city_market_id   | permanent   | btree         | 144 kB |
 public | pk_l_airport             | index | postgres | l_airport_id       | permanent   | btree         | 160 kB |
 public | pk_l_unique_carriers     | index | postgres | l_unique_carriers  | permanent   | btree         | 72 kB  |
 public | pk_weekdays              | index | postgres | l_weekdays         | permanent   | btree         | 16 kB  |
(5 rows)
```

**Add PKs & FKs**

```
                          Table "public.t_ontime"
        Column          |          Type          | Collation | Nullable | Default
----------------------+----------------------+-----------+----------+---------
 year                 | integer              |           |          |
 quarter              | integer              |           |          |
 month                | integer              |           |          |
 day_of_month         | integer              |           |          |
...

Indexes:
    "oco_market_desc_fl_date" btree (origin_city_market_id, fl_date)
Foreign-key constraints:
    "fk_airline_id" FOREIGN KEY (origin_airport_id) REFERENCES l_airport_id(code)
    "fk_dest_airline_id" FOREIGN KEY (dest_airport_id) REFERENCES l_airport_id(code)
    "fk_dest_city" FOREIGN KEY (dest_city_market_id) REFERENCES l_city_market_id(code)
    "fk_origin_city" FOREIGN KEY (origin_city_market_id) REFERENCES l_city_market_id(code)
    "fk_unique_carrier" FOREIGN KEY (unique_carrier) REFERENCES l_unique_carriers(code)
```

# ERD with Added Constraints https://www.pgadmin.org/download/



```
                                    QUERY PLAN
----------------------------------------------------------------------------------------------------------------------------------
Nested Loop  (cost=15.89..3184.20 rows=5 width=263) (actual time=0.359..1.710 rows=62 loops=1)
  Buffers: shared hit=907
  ->  Nested Loop  (cost=15.60..3182.69 rows=5 width=250) (actual time=0.352..1.589 rows=62 loops=1)
        Buffers: shared hit=721
        ->  Nested Loop  (cost=15.32..3181.19 rows=5 width=217) (actual time=0.344..1.466 rows=62 loops=1)
              Buffers: shared hit=535
              ->  Nested Loop  (cost=15.04..3179.68 rows=5 width=184) (actual time=0.333..1.350 rows=62 loops=1)
                    Buffers: shared hit=349
                    ->  Nested Loop  (cost=14.76..3178.20 rows=5 width=167) (actual time=0.312..0.989 rows=62 loops=1)
                          Join Filter: (w.code = o.day_of_week)
                          Rows Removed by Join Filter: 358
                          Buffers: shared hit=163
                          ->  Seq Scan on l_weekdays w  (cost=0.00..1.10 rows=1 width=122) (actual time=0.016..0.017 rows=1 loops=1)
                                Filter: ((description)::text = 'Friday'::text)
                                Rows Removed by Filter: 7
                                Buffers: shared hit=1
                          ->  Nested Loop  (cost=14.76..3176.60 rows=40 width=53) (actual time=0.291..0.937 rows=420 loops=1)
                                Buffers: shared hit=162
                                ->  Seq Scan on l_city_market_id co  (cost=0.00..110.79 rows=1 width=21) (actual time=0.237..0.617 rows=1 loops=1)
                                      Filter: ((description)::text = 'Little Rock, AR'::text)
                                      Rows Removed by Filter: 5822
                                      Buffers: shared hit=38
                                ->  Bitmap Heap Scan on t_ontime o  (cost=14.76..3057.72 rows=810 width=40) (actual time=0.049..0.210 rows=420 loops=1)
                                      Recheck Cond: ((origin_city_market_id = co.code)
                                            AND (fl_date >= '2015-02-01'::date) AND (fl_date <= '2015-02-16'::date))
                                      Heap Blocks: exact=120
                                      Buffers: shared hit=124
                                      ->  Bitmap Index Scan on oco_market_desc_fl_date  (cost=0.00..14.56 rows=810 width=0) (actual time=0.030..0.030 rows=420 loops=1)
                                            Index Cond: ((origin_city_market_id = co.code)
                                                  AND (fl_date >= '2015-02-01'::date) AND (fl_date <= '2015-02-16'::date))
                                            Buffers: shared hit=4
                    ->  Index Scan using pk_l_unique_carriers on l_unique_carriers uc  (cost=0.28..0.30 rows=1 width=31) (actual time=0.005..0.005 rows=1 loops=62)
                          Index Cond: (code = (o.unique_carrier)::bpchar)
                          Buffers: shared hit=186
              ->  Index Scan using pk_l_airport on l_airport_id ao  (cost=0.28..0.30 rows=1 width=41) (actual time=0.001..0.001 rows=1 loops=62)
                    Index Cond: (code = o.origin_airport_id)
                    Buffers: shared hit=186
        ->  Index Scan using pk_l_airport on l_airport_id ad  (cost=0.28..0.30 rows=1 width=41) (actual time=0.002..0.002 rows=1 loops=62)
              Index Cond: (code = o.dest_airport_id)
              Buffers: shared hit=186
  ->  Index Scan using pk_city_market on l_city_market_id cd  (cost=0.28..0.30 rows=1 width=21) (actual time=0.001..0.001 rows=1 loops=62)
        Index Cond: (code = o.dest_city_market_id)
        Buffers: shared hit=186
Planning:
  Buffers: shared hit=8
Planning Time: 2.462 ms
Execution Time: 1.818 ms
```

4.28 ms

Previous> 41.047 ms 326.229 ms 2.07 S

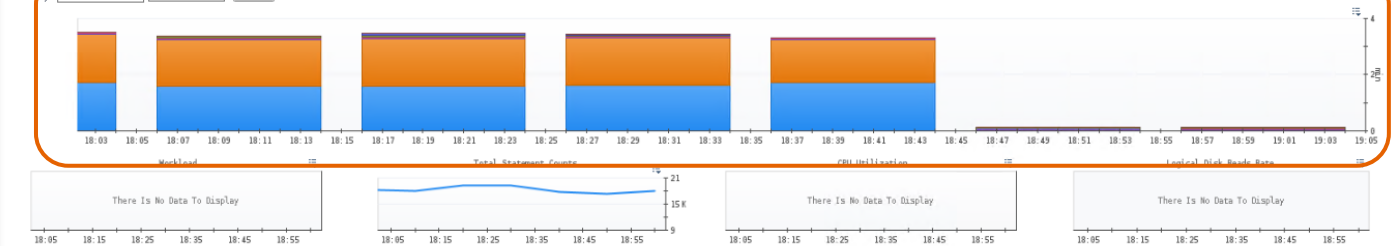
# Best Average Time

Databases > PostgreSQL Server: jgss2017.internal.cloudapp.net:5432 > Statements for Postgresqlagent

Tuesday, Apr 2, 2024 06:02:09 PM - Now 1 hour | Reports

Overview | Databases | Tables | Connections | BG Writer | Locks | Configuration | **Statements** | Tablespaces

Order By: Total Time 10 Apply




Search

| Statement | | | | | | | Memory Blocks | | | | Shared Blocks |
|---|---|---|---|---|---|---|---|---|---|---|---|
| User | DB | Query | Calls | Average | Total | Avg Rows | Write Time | Read Time | Read Time per Query | Read Time per Row | Hit Pct |
| postgres | test | PREPARE billing (timestamp,timestamp) as SELECT s.fname, s.lname, ... | 46.3 K | 0.40 ms | 18.7 sec | 52.3 | 0 ms | 0 ms | 0.00 ms | 0.00 ms | 100.00 % |
| postgres | dot | fetch all in "pop_flights" | 441 | 1.44 s | 10.6 min | 1,069.6 | 0 ms | 0 ms | 0.00 ms | 0.00 ms | 100.00 % |
| postgres | dot | SELECT o.carrier, uc.description AS carrier_name, o.fl_date,o.fl_num... | 440 | 1.45 s | 10.7 min | 1,068.2 | 0 ms | 0 ms | 0.00 ms | 0.00 ms | 100.00 % |
| postgres | dot | CALL public.get_pop_flight('Denver, CO','2015-02-01','2015-02-16','T... | 56 | 0.15 ms | 8.5 ms | 0 | 0 ms | 0 ms | 0.00 ms | 0.00 ms | |
| postgres | dot | CALL public.get_pop_flight('Denver, CO','2015-02-01','2015-02-16','M... | 56 | 0.30 ms | 16.8 ms | 0 | 0 ms | 0 ms | 0.00 ms | 0.00 ms | 100.00 % |
| postgres | dot | CALL public.get_pop_flight('Denver, CO','2015-02-01','2015-02-16','W... | 56 | 0.41 ms | 23.2 ms | 0 | 0 ms | 0 ms | 0.00 ms | 0.00 ms | 100.00 % |
| postgres | dot | CALL public.get_pop_flight('Denver, CO','2015-02-01','2015-02-16','Sa... | 55 | 0.40 ms | 21.8 ms | 0 | 0 ms | 0 ms | 0.00 ms | 0.00 ms | 98.61 % |
| postgres | dot | CALL public.get_pop_flight('Denver, CO','2015-02-01','2015-02-16','S... | 54 | 0.16 ms | 8.4 ms | 0 | 0 ms | 0 ms | 0.00 ms | 0.00 ms | n/a |
| postgres | dot | CALL public.get_pop_flight('Denver, CO','2015-02-01','2015-02-16','T... | 54 | 0.29 ms | 15.9 ms | 0 | 0 ms | 0 ms | 0.00 ms | 0.00 ms | 97.22 % |
| postgres | dot | CALL public.get_pop_flight('Denver, CO','2015-02-01','2015-02-16','Fr... | 53 | 0.60 ms | 31.7 ms | 0 | 0 ms | 0 ms | 0.00 ms | 0.00 ms | 99.39 % |
| postgres | postgres | SELECT name, setting, unit from pg_settings | 36 | 12.04 ms | 433.5 ms | 365 | 0 ms | 0 ms | 0.00 ms | 0.00 ms | n/a |
| postgres | postgres | SELECT i.relid as table_id, i.indexrelid as indexrelid, i.indexrelname as... | 13 | 0.49 ms | 6.4 ms | 0 | 0 ms | 0 ms | 0.00 ms | 0.00 ms | 100.00 % |
| postgres | dot | SELECT funcid, schemaname, funcname, calls, total_time, self_time F... | 13 | 1.62 ms | 21.1 ms | 3 | 0 ms | 0 ms | 0.00 ms | 0.00 ms | 100.00 % |
| postgres | postgres | SELECT slot_name, plugin, slot_type, database, active, pid, usesysid,... | 12 | 0.29 ms | 3.5 ms | 0 | 0 ms | 0 ms | 0.00 ms | 0.00 ms | n/a |
| postgres | postgres | SELECT t.relid as relid, t.schemaname as schema, t.relname as name, ... | 12 | 0.34 ms | 4.1 ms | 0 | 0 ms | 0 ms | 0.00 ms | 0.00 ms | 100.00 % |
| postgres | postgres | SELECT *, EXTRACT($1 FROM (current_timestamp - least(query_sta... | 12 | 0.42 ms | 5.1 ms | 14 | 0 ms | 0 ms | 0.00 ms | 0.00 ms | 100.00 % |
| postgres | postgres | SELECT pg_locks.pid as pid, locktype, virtualtransaction, transactio... | 12 | 2.34 ms | 28.1 ms | 58.3 | 0 ms | 0 ms | 0.00 ms | 0.00 ms | 100.00 % |
| postgres | postgres | SELECT s.queryid as queryid, s.userid as userid, s.dbid as dbid, s.cal... | 12 | 8.37 ms | 100.5 ms | 300 | 0 ms | 0 ms | 0.00 ms | 0.00 ms | 100.00 % |
| postgres | postgres | SELECT d.datid as dbid, d.datname as name, t.spcname as tablespac... | 12 | 1.39 s | 16.7 sec | 4 | 0 ms | 0 ms | 0.00 ms | 0.00 ms | 100.00 % |
| postgres | postgres | SELECT t.oid as oid, spcname as name, u.usename as owner, pg_tabl... | 12 | 1.93 s | 23.1 sec | 2 | 0 ms | 0 ms | 0.00 ms | 0.00 ms | 100.00 % |
| postgres | dot | SELECT a.attname, pg_catalog.format_type(a.atttypid, a.atttypmod),... | 10 | 0.62 ms | 6.2 ms | 7.7 | 0 ms | 0 ms | 0.00 ms | 0.00 ms | 99.67 % |
| postgres | dot | SELECT c.relchecks, c.relkind, c.relhasindex, c.relhasrules, c.relhastr... | 9 | 0.07 ms | 634.4 µs | 1 | 0 ms | 0 ms | 0.00 ms | 0.00 ms | 100.00 % |
| postgres | dot | CALL public.get_pop_flight('Little Rock, AR','2015-02-01','2015-02-16',... | 8 | 0.16 ms | 1.2 ms | 0 | 0 ms | 0 ms | 0.00 ms | 0.00 ms | n/a |
| postgres | dot | CALL public.get_pop_flight('Little Rock, AR','2015-02-01','2015-02-16',... | 8 | 0.16 ms | 1.2 ms | 0 | 0 ms | 0 ms | 0.00 ms | 0.00 ms | n/a |

# Entire Tuning Effects on Workload

Search

| Statement | | | | | | | Memory Blocks | | | | Shared Blocks |
|---|---|---|---|---|---|---|---|---|---|---|---|
| User | DB | Query | Calls | Average | Total | Avg Rows | Write Time | Read Time | Read Time per Query | Read Time per Row | Hit Pct |
| postgres | test | PREPARE billing (timestamp,timestamp) as SELECT s.fname, s.lname, ... | 230.7 K | 0.41 ms | 1.6 min | 52.2 | 0 ms | 0 ms | 0.00 ms | 0.00 ms | 100.00 % |
| postgres | dot | fetch all in "pop_flights" | 2.8 K | 3.44 s | 2.7 hr | 553.4 | 0 ms | 0 ms | 0.00 ms | 0.00 ms | 72.40 % |
| postgres | dot | SELECT o.carrier, uc.description AS carrier_name, o.fl_date,o.fl_num... | 2.8 K | 3.44 s | 2.7 hr | 552.6 | 0 ms | 0 ms | 0.00 ms | 0.00 ms | 72.32 % |
| postgres | dot | SELECT funcid, schemaname, funcname, calls, total_time, self_time F... | 441 | 1.57 ms | 690.6 ms | 0.5 | 0 ms | 0 ms | 0.00 ms | 0.00 ms | 76.96 % |
| postgres | dot | CALL public.get_pop_flight('Little Rock, AR','2015-02-01','2015-02-16',... | 233 | 0.11 ms | 25 ms | 0 | 0 ms | 0 ms | 0.00 ms | 0.00 ms | n/a |
| postgres | dot | CALL public.get_pop_flight('Little Rock, AR','2015-02-01','2015-02-16',... | 233 | 0.14 ms | 31.5 ms | 0 | 0 ms | 0 ms | 0.00 ms | 0.00 ms | n/a |
| postgres | dot | CALL public.get_pop_flight('Little Rock, AR','2015-02-01','2015-02-16',... | 233 | 0.14 ms | 31.6 ms | 0 | 0 ms | 0 ms | 0.00 ms | 0.00 ms | n/a |
| postgres | dot | CALL public.get_pop_flight('Little Rock, AR','2015-02-01','2015-02-16',... | 233 | 0.14 ms | 31.8 ms | 0 | 0 ms | 0 ms | 0.00 ms | 0.00 ms | n/a |
| postgres | dot | CALL public.get_pop_flight('Little Rock, AR','2015-02-01','2015-02-16',... | 233 | 0.14 ms | 31.9 ms | 0 | 0 ms | 0 ms | 0.00 ms | 0.00 ms | n/a |
| postgres | dot | CALL public.get_pop_flight('Little Rock, AR','2015-02-01','2015-02-16',... | 233 | 0.16 ms | 36.9 ms | 0 | 0 ms | 0 ms | 0.00 ms | 0.00 ms | n/a |
| postgres | dot | CALL public.get_pop_flight('Little Rock, AR','2015-02-01','2015-02-16',... | 233 | 0.56 ms | 129.3 ms | 0 | 0 ms | 0 ms | 0.00 ms | 0.00 ms | 100.00 % |
| postgres | postgres | SELECT name, setting, unit from pg_settings | 178 | 12.26 ms | 2.2 sec | 365 | 0 ms | 0 ms | 0.00 ms | 0.00 ms | n/a |
| postgres | dot | CALL public.get_pop_flight('Denver, CO','2015-02-01','2015-02-16','T... | 172 | 0.18 ms | 30.4 ms | 0 | 0 ms | 0 ms | 0.00 ms | 0.00 ms | 72.00 % |
| postgres | dot | CALL public.get_pop_flight('Denver, CO','2015-02-01','2015-02-16','W... | 172 | 0.65 ms | 112.6 ms | 0 | 0 ms | 0 ms | 0.00 ms | 0.00 ms | 78.68 % |
| postgres | dot | CALL public.get_pop_flight('Denver, CO','2015-02-01','2015-02-16','S... | 171 | 0.14 ms | 23.1 ms | 0 | 0 ms | 0 ms | 0.00 ms | 0.00 ms | n/a |
| postgres | dot | CALL public.get_pop_flight('Denver, CO','2015-02-01','2015-02-16','T... | 171 | 0.18 ms | 30.8 ms | 0 | 0 ms | 0 ms | 0.00 ms | 0.00 ms | 97.22 % |
| postgres | dot | CALL public.get_pop_flight('Denver, CO','2015-02-01','2015-02-16','Fr... | 171 | 0.40 ms | 69.1 ms | 0 | 0 ms | 0 ms | 0.00 ms | 0.00 ms | 91.37 % |
| postgres | dot | CALL public.get_pop_flight('Denver, CO','2015-02-01','2015-02-16','M... | 171 | 164.95 ms | 28.2 sec | 0 | 0 ms | 0 ms | 0.00 ms | 0.00 ms | 95.95 % |
| postgres | dot | CALL public.get_pop_flight('Denver, CO','2015-02-01','2015-02-16','Sa... | 170 | 22.75 ms | 3.9 sec | 0 | 0 ms | 0 ms | 0.00 ms | 0.00 ms | 96.42 % |
| postgres | postgres | SELECT pg_locks.pid as pid, locktype, virtualtransaction, transactio... | 60 | 1.79 ms | 107.5 ms | 45.5 | 0 ms | 0 ms | 0.00 ms | 0.00 ms | 98.76 % |
| postgres | postgres | SELECT slot_name, plugin, slot_type, database, active, pid, usesysid,... | 59 | 0.28 ms | 16.7 ms | 0 | 0 ms | 0 ms | 0.00 ms | 0.00 ms | n/a |
| postgres | postgres | SELECT s.queryid as queryid, s.userid as userid, s.dbid as dbid, s.cal... | 59 | 8.99 ms | 530.6 ms | 300 | 0 ms | 0 ms | 0.00 ms | 0.00 ms | 100.00 % |
| postgres | postgres | SELECT d.datid as dbid, d.datname as name, t.spcname as tablespac... | 59 | 1.45 s | 1.4 min | 4 | 0 ms | 0 ms | 0.00 ms | 0.00 ms | 100.00 % |
| postgres | postgres | SELECT t.oid as oid, spcname as name, u.usename as owner, pg_tabl... | 59 | 1.96 s | 1.9 min | 2 | 0 ms | 0 ms | 0.00 ms | 0.00 ms | 100.00 % |

# Summary

- **Monitor Wait time**
- **Review the Execution Plan**
  - Look for Costly Steps
- **Gather Object Info**
- **Find the Driving Table**
- **Engineer out the Stupid**
  - Common mistakes
- **Compare your Tuning Results**
  - Brag about Yourself … No one else will!

# References & Additional Info

- PostgreSQL Query Optimization - The Ultimate Guide...
- PostgreSQL Cheatsheet
- Dynamic & Collected Statistics Collection Configuration
- Working With Postgres WAL Made Easy 101
- Autovacuum Configuration Information
- Routine Vacuuming & Analyze
- Appendix
  - More Stupid Mistakes
  - More Explain Common Mistakes
  - Examine the Explain Plan of OLAP Queries

Thank YOU!!!

# More Stupid Mistakes

- **Not using appropriate indexes leads to slow query & increased database workload**
  - Try using SQL Diagraming techniques to find the best index to drive the least amount of data required
  - Too many indexes can increase the time spent on DML operations
  - Using the wrong type of index - e.g. B-tree index for Full-text search queries
- **Data type mismatch**
  - Comparing columns with wrong data types can lead to errors or incorrect results (implicit conversions)
  - Make sure that the data types of the columns being compared or combined in the query are compatible
- **Not utilizing referential integrity**
  - Generic table design (i.e. PK, tag value)
  - No primary or foreign keys, etc…
- **Common Postgres Beginner Mistakes & Best Practices**
- **Don't Do This! (PostgreSQL Mistakes & How to Avoid Them**

# Explain Plan - Look for Common Mistakes

- **Identify Common Mistakes**
  - Using functions on indexed columns
    - In WHERE, ON & HAVING clause
    - Create a Functional Index instead
      - Create index lower_title_idx on film(lower(title));
  - Nested views
    - One view calling or joining to other views
  - Use of cursors or row by row processing
- **Missing or Poor Indexing**
- **Problems Outside of the Plan**
  - Missing or stale statistics
  - Database misconfiguration
  - No database constraints

# Examine the Explain Plan of OLAP Queries

- **Consider using set operations**
  - Instead of NOT EXISTS or NOT IN – **use EXCEPT**
  - Instead of EXISTS or IN – **use INTERSECT**
  - Instead of complex OR - **use UNION**
  - Review join order, try for most selective join first
- **Avoid multiple scans on same table**
  - Sometimes a design issue where generic table is designed
    - PK, tag, value
- **Consider Temp tables, CTEs & Materialized views on OLAP queries to speed them up**
  - No indexes or statistics on temp tables – be careful of large temp tables
  - Can cause excessive I/O because of writes to disk
  - CTEs can prevent Optimizer from choosing join order
  - *Dombrovskaya, Henrietta; Bailliekova, Anna. PostgreSQL Query Optimization: The Ultimate Guide to Building Efficient Queries (p. 121). Apress. Kindle Edition.*